

ROS 2-Robotik-Handbuch

Theoretische Grundlagen, Hardware, Simulation und vier Projekte

Stepper-Robotik mit Zykloidgetriebe · Stereokamera · Kalman-Filter · ROS 2

Arbeitsdokument

Stand: Juni 2026

ROS 2 Jazzy Jalisco / Gazebo Harmonic

Über dieses Dokument

Wie dieses Dokument zu lesen ist

Das Handbuch ist in sechs Teile gegliedert. **Teil I** legt die Theorie fest (ROS2, Kalman/EKF, Odometrie, Stereosehen, Kinematik, Dynamik, Trajektorien, Schrittmotoren, Zykloidgetriebe). **Teil II** behandelt die Hardware-Auswahl inklusive Strom-/Spannungsmessung zur Drehmomentschätzung und PCB-Optimierung. **Teil III** ist die Schritt-für-Schritt-Installationsanleitung (VM, Linux, ROS2). **Teil IV** erklärt die Simulation, den CAD-zu-URDF-Workflow und die entscheidende Frage, wo sich Simulation und reale Implementierung trennen. **Teil V** arbeitet die vier Projekte mit Knoten-Architektur und `rqt_graph` durch. **Teil VI** ist die praktische Stereokamera-Integration mit Codebeispielen. Den Abschluss bildet eine empfohlene Dokumentationsmethodik.

Inhaltsverzeichnis

Über dieses Dokument	i
I Theoretische Grundlagen	1
1 ROS 2 – Konzepte und Architektur	2
1.1 Warum ROS 2 und nicht ROS 1	2
1.2 Die fünf Kommunikationsprimitive	2
1.3 QoS – Quality of Service	3
1.4 Workspace, Pakete, Build	3
1.5 TF2 – der Transformationsbaum	3
1.6 Wichtige CLI-Werkzeuge	3
2 Zustandsschätzung: Kalman-Filter und EKF	5
2.1 Das Grundproblem	5
2.2 Lineares Zustandsraummodell	5
2.3 Die zwei Phasen	5
2.4 Extended Kalman Filter (EKF)	6
2.5 Anwendung Tischtennisball (konstante Beschleunigung)	6
3 Odometrie und Sensorfusion	7
4 Stereosehen und Merkmalerkennung	8
4.1 Lochkameramodell und Tiefe aus Disparität	8
4.2 Kalibrierung	8
4.3 Merkmalerkennung ohne eigenes KI-Training	8
4.4 Posenschätzung – 6D-Lage eines Bauteils	9
5 6-DoF-Posenschätzung mit gelernten Surface Embeddings	10
5.1 Das Problem: sechs Freiheitsgrade	10
5.2 Rotationsrepräsentationen	10
5.3 Warum gelernte Embeddings statt Regression	11
5.4 Kontrastives Lernen der Embeddings	11
5.5 Das Two-Tower-Modell (Query-Key)	11
5.6 Von Korrespondenzen zur Pose: PnP + RANSAC	12
5.7 Registrierung: grob und fein	12
5.8 Punktwolken-Netze: PointNet und PointNet++	13
5.9 Stereo, Tiefe und Skalierung	13
5.10 Zeitliche Konsistenz: Kalman-Pose-Tracking	13
5.11 Sim-to-Real und Trainingsdaten	13
5.12 Benchmarking und Metriken	13

6	Kinematik, Dynamik und Trajektorien	15
6.1	Vorwärtskinematik	15
6.2	Inverse Kinematik	15
6.3	Dynamik	15
6.4	Trajektorienplanung	15
7	Schrittmotoren und Drehmomentschätzung	16
7.1	NEMA-17-Grundlagen	16
7.2	Indirekte Drehmomentschätzung	16
8	Zykloidgetriebe	17
II	Hardware-Auswahl	18
9	Komponenten im günstigen Hobby-Bereich	19
9.1	Schrittmotor und Treiber	19
9.2	Recheneinheiten	19
9.3	Stereokamera	20
10	Optimierungspotenzial für ein eigenes PCB	21
III	Software-Setup: VM, Linux, ROS 2	22
11	Betriebssystem und Distribution	23
11.1	Distributionswahl (Stand Juni 2026)	23
11.2	VM, Dual-Boot oder WSL2?	23
12	ROS 2 installieren – Schritt für Schritt	24
12.1	Pakete, die du für die Projekte brauchst	24
IV	Simulation und der Weg vom CAD-Modell zur Realität	26
13	Simulationswerkzeuge im ROS 2-Ökosystem	27
13.1	Was lässt sich vorab simulieren?	27
13.2	MuJoCo und Isaac Sim im Detail	28
13.2.1	MuJoCo – Multi-Joint dynamics with Contact	28
13.2.2	NVIDIA Isaac Sim – photorealistisch und GPU-parallel	28
13.3	Lassen sie sich integriert einsetzen?	29
14	Wo sich Simulation und reale Implementierung trennen	30
15	Vorabschritte: vom CAD-Modell zur Simulation	31
15.1	Getrennte STL-Exporte pro Gelenk	31
15.2	Gelenktypen	32
15.3	Getriebeübersetzung in ros2_control	32
V	Die vier Projekte: Architektur und rqt_graph	33
16	Projekt 1 – Stepper-Roboter mit Trajektorienbefehl vom PC	34
16.1	ROS 2-Bausteine	34
16.2	rqt_graph (vereinfacht)	34

17 Projekt 2 – Tischtennisroboter	35
17.1 ROS 2-Bausteine	35
17.2 rqt_graph (vereinfacht)	35
18 Projekt 3 – Mobiler Roboter: Odometrie, Navigation, Bewegungserkennung	36
18.1 ROS 2-Bausteine	36
19 Projekt 4 – CAD-basierte Posenerkennung und Greifen	37
19.1 ROS 2-Bausteine	37
19.2 Zwei Wege zur 6D-Pose	37
19.3 Inferenz-Pipeline (lernbasiert)	38
19.4 Stereo als metrische Skala	38
19.5 Bewegte Teile: Pose-Tracking mit Kalman	38
19.6 Hyperparameter und Evaluation	38
VI Stereokamera in ROS 2 – Praxis mit Code	40
20 Kamera anbinden, Frame splitten	41
21 Kalibrieren und Tiefe erzeugen	42
22 Kalman-Filter-Knoten (Ballverfolgung)	43
23 Merkmalerkennung ohne KI-Training	44
24 Echtzeit-Stereoverarbeitung: Scheduling, Frame-Dropping und Zustandsauto-	45
mat	
24.1 Das Zeitbudget eines Frames	45
24.2 Frame-Dropping: DDS macht die Arbeit	46
24.3 Scheduling: Executor und Callback-Gruppen	46
24.4 Ablauflogik als Zustandsautomat	47
VII Tischtennisroboter: Mechanik, Kamera-Architektur und CAD/URDF	49
25 Kamera-Architektur – wo wird gerechnet?	50
25.1 Die Bandbreitenrechnung entscheidet	50
25.2 Verdikt pro Plattform	50
25.3 Sparse statt dense: der Ball braucht keine volle Tiefenkarte	51
25.4 ROS-Service, UDP oder Topic?	51
26 Mechanischer Aufbau: Portalsystem (Gantry)	52
26.1 Grundkonzept	52
26.2 Geschachtelte Achsen	52
26.3 Antriebstopologie und Hardware-Anschluss	52
27 Das Hebelarm- und Schwingungsproblem	54
27.1 Die Physik	54
27.2 Gegenmaßnahmen	54
28 Simulierbarkeit in Gazebo und RViz	56
28.1 Was Gazebo kann – und was nicht	56
28.2 Wege, die Nachgiebigkeit doch abzubilden	56

29 CAD-Module und URDF	58
29.1 Notwendige CAD-Module mit Designanforderungen	58
29.2 Kinematische Kette	58
29.3 URDF/Xacro-Gerüst	58
 VIII Dokumentationsmethodik	 60
 30 Vorgehensweise festhalten	 61
 Literaturverzeichnis	 62

Teil I

Theoretische Grundlagen

Kapitel 1

ROS 2 – Konzepte und Architektur

1.1 Warum ROS 2 und nicht ROS 1

ROS 2 ersetzt den zentralen `rosmaster` von ROS 1 durch eine dezentrale Discovery über **DDS** (Data Distribution Service), einen Industriestandard aus Luft- und Raumfahrt. Daraus folgen die für deine Projekte relevanten Eigenschaften: kein Single-Point-of-Failure, Multi-Maschinen-Betrieb über das Netzwerk ohne Master, konfigurierbare Echtzeit-/Zuverlässigkeitsgarantien (QoS) und ein einheitliches API in C++ (`rclcpp`) und Python (`rclpy`). ROS 1 (Noetic) ist seit Mai 2025 End-of-Life; alle Neuprojekte gehören auf ROS 2.

1.2 Die fünf Kommunikationsprimitive

Das gesamte Verhalten eines ROS 2-Systems lässt sich auf fünf Bausteine zurückführen. Sie zu verstehen ist die halbe Miete – der Rest ist Anwendung.

Node (Knoten)

Eine Recheneinheit mit genau einer Verantwortung – z. B. `ball_detector`, `trajectory_controller`. Knoten kommunizieren ausschließlich über die vier folgenden Mechanismen, nie direkt.

Topic (Thema)

Asynchroner Publish/Subscribe-Kanal, many-to-many. Ein Publisher sendet *Nachrichten* (eines `.msg`-Typs), beliebig viele Subscriber empfangen sie. Geeignet für kontinuierliche Datenströme: Bilder, Gelenkzustände, Odometrie. *Feuern und vergessen* – der Sender weiß nicht, wer zuhört.

Service (Dienst)

Synchroner Request/Response, one-to-one. Ein Client schickt eine Anfrage, ein Server antwortet genau einmal. Für kurze, abgeschlossene Aktionen: “Schalte Controller um”, “Gib aktuelle Kalibrierung zurück”. Blockierend.

Action (Aktion)

Für lange, zielgerichtete Aufgaben mit Zwischen-Feedback und Abbruchmöglichkeit. Technisch aus Topics + Services zusammengesetzt: *Goal* (Ziel senden), *Feedback* (Fortschritt), *Result* (Endergebnis). Der Paradefall für “fahre diese Trajektorie ab” – du willst Fortschritt sehen und abbrechen können.

Parameter

Pro Knoten gehaltene Konfigurationswerte (Gelenklimits, PID-Gains, Getriebeübersetzung), zur Laufzeit les- und setzbar, deklariert beim Knotenstart.

Faustregel zur Wahl

Kontinuierlicher Datenstrom → **Topic**. Kurze Frage mit sofortiger Antwort → **Service**. Langer Auftrag mit Fortschritt/Abbruch (Trajektorie, Navigation, Greifen) → **Action**. Statische Konfiguration → **Parameter**.

1.3 QoS – Quality of Service

Weil DDS darunterliegt, hat jedes Topic eine QoS-Konfiguration. Die wichtigsten Profile:

- **Reliability:** RELIABLE (jede Nachricht garantiert, Wiederübertragung) vs. BEST_EFFORT (verlustbehaftet, aber latenzarm). Sensorbilder fahren oft BEST_EFFORT, Steuerbefehle RELIABLE.
- **Durability:** TRANSIENT_LOCAL hält die letzte Nachricht für später startende Subscriber vor (typisch für /map, statische TF).
- **History/Depth:** Puffergröße.

Eine häufige Fehlerquelle: Publisher und Subscriber mit *inkompatiblen* QoS-Profilen “sehen” sich nicht. Bei stummen Topics ist das der erste Verdacht.

1.4 Workspace, Pakete, Build

ROS 2-Code lebt in einem **Workspace** (Verzeichnis mit `src/`). Darin liegen **Pakete**. Gebaut wird mit **colcon**.

```
1 mkdir -p ~/ros2_ws/src && cd ~/ros2_ws
2 # ... Pakete nach src/ ...
3 rosdep install --from-paths src --ignore-src -y      # Abhaengigkeiten
4 colcon build --symlink-install                     # bauen
5 source install/setup.bash                          # Overlay aktivieren
```

Listing 1.1: Workspace anlegen und bauen

Python-Pakete nutzen `ament_python` (`setup.py`), C++-Pakete `ament_cmake` (`CMakeLists.txt`). Beide haben eine `package.xml` mit Metadaten und Abhängigkeiten.

1.5 TF2 – der Transformationsbaum

Jeder Roboter ist ein Baum von Koordinatensystemen (Frames): `map` → `odom` → `base_link` → `camera_link` → **TF2** verwaltet diese zeitgestempelten Transformationen, sodass du jederzeit fragen kannst “Wo ist der Ball im `base_link`-Frame?”, obwohl er in `camera_link` gemessen wurde. Rotationen werden hier als **Quaternionen** gespeichert (`geometry_msgs/Quaternion`) – genau aus den Gründen, die wir besprochen haben: kein Gimbal Lock, kompakt, sauber interpolierbar. Konventionen sind in REP-103 (Einheiten, rechtshändig, *x* vorne, *z* oben) und REP-105 (Frame-Namen) festgelegt.

1.6 Wichtige CLI-Werkzeuge

Befehl	Zweck
<code>ros2 run <pkg> <node></code>	Einen Knoten starten
<code>ros2 launch <pkg> <file></code>	Mehrere Knoten + Konfiguration starten
<code>ros2 topic list / echo / hz</code>	Topics auflisten, Inhalt sehen, Rate messen
<code>ros2 node list / info</code>	Knoten und ihre Schnittstellen

<code>ros2 service call</code>	Dienst manuell aufrufen
<code>ros2 action send_goal</code>	Aktion manuell starten
<code>ros2 param get / set / dump</code>	Parameter lesen/schreiben/sichern
<code>rqt_graph</code>	Knoten-Topic-Graph visualisieren
<code>rviz2</code>	3D-Visualisierung (<i>kein</i> Simulator!)
<code>ros2 bag record / play</code>	Datenströme aufzeichnen/abspielen

Häufige Begriffsverwechslung

RViz2 ist ein *Visualisierer* (zeigt Daten an), **Gazebo** ist ein *Simulator* (erzeugt Daten per Physik). Du wirst beide gleichzeitig nutzen: Gazebo simuliert, RViz2 zeigt an.

Kapitel 2

Zustandsschätzung: Kalman-Filter und EKF

2.1 Das Grundproblem

Du willst eine verborgene Größe (Position und Geschwindigkeit des Tischtennisballs, Pose des Roboters) aus verrauschten, unvollständigen Messungen schätzen – und das, während sich der Zustand zeitlich ändert. Der Kalman-Filter ist der optimale lineare Schätzer für genau dieses Problem unter gaußischem Rauschen. Er hält nicht nur einen Schätzwert $\hat{\mathbf{x}}$, sondern auch dessen **Unsicherheit** als Kovarianzmatrix $\mathbf{P}$.

2.2 Lineares Zustandsraummodell

$$\mathbf{x}_k = \mathbf{F} \mathbf{x}_{k-1} + \mathbf{B} \mathbf{u}_k + \mathbf{w}_k, \quad \mathbf{w}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}) \quad (\text{Prozessrauschen}) \quad (2.1)$$

$$\mathbf{z}_k = \mathbf{H} \mathbf{x}_k + \mathbf{v}_k, \quad \mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R}) \quad (\text{Messrauschen}) \quad (2.2)$$

$\mathbf{F}$ = Systemmatrix (wie entwickelt sich der Zustand), $\mathbf{B}$ = Steuermatrix, $\mathbf{H}$ = Messmatrix (was misst der Sensor vom Zustand), $\mathbf{Q}/\mathbf{R}$ = Rausch-Kovarianzen (deine wichtigsten Tuning-Größen).

2.3 Die zwei Phasen

Der Filter alterniert zwischen **Prädiktion** (Modell rechnet voraus, Unsicherheit wächst) und **Korrektur** (Messung zieht zurück, Unsicherheit sinkt).

Prädiktion (Zeitschritt)

$$\begin{aligned} \hat{\mathbf{x}}_k^- &= \mathbf{F} \hat{\mathbf{x}}_{k-1} + \mathbf{B} \mathbf{u}_k \\ \mathbf{P}_k^- &= \mathbf{F} \mathbf{P}_{k-1} \mathbf{F}^\top + \mathbf{Q} \end{aligned}$$

Korrektur (Messupdate)

$$\begin{aligned} \mathbf{K}_k &= \mathbf{P}_k^- \mathbf{H}^\top (\mathbf{H} \mathbf{P}_k^- \mathbf{H}^\top + \mathbf{R})^{-1} && (\text{Kalman-Gain}) \\ \hat{\mathbf{x}}_k &= \hat{\mathbf{x}}_k^- + \mathbf{K}_k (\mathbf{z}_k - \mathbf{H} \hat{\mathbf{x}}_k^-) && (\text{Innovation gewichtet}) \\ \mathbf{P}_k &= (\mathbf{I} - \mathbf{K}_k \mathbf{H}) \mathbf{P}_k^- \end{aligned}$$

Der Gain $\mathbf{K}$ ist die Gewichtung: Ist die Messung präzise ($\mathbf{R}$ klein), vertraut der Filter ihr; ist das Modell präzise ($\mathbf{Q}$ klein), glättet er stärker.

2.4 Extended Kalman Filter (EKF)

Sobald System oder Messung **nichtlinear** sind – und das sind sie in der Robotik fast immer (Rotationen, Projekttilflug mit Luftwiderstand, Kameraprojektion) –, ersetzt man die Matrizen durch nichtlineare Funktionen $\mathbf{f}(\cdot)$ und $\mathbf{h}(\cdot)$ und linearisiert sie pro Schritt über die **Jacobi-Matrizen**:

$$\mathbf{F}_k = \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_{k-1}}, \quad \mathbf{H}_k = \left. \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_k^-} \quad (2.3)$$

Prädiktion nutzt dann $\hat{\mathbf{x}}_k^- = \mathbf{f}(\hat{\mathbf{x}}_{k-1}, \mathbf{u}_k)$, die Kovarianz-Updates verwenden $\mathbf{F}_k, \mathbf{H}_k$. Der **UKF** (Unscented) vermeidet die Jacobi-Matrizen, indem er “Sigma-Punkte” durch die Nichtlinearität propagiert – genauer bei starker Krümmung, aber rechenintensiver. Für Lageschätzung mit Quaternionen nimmt man den **Error-State/Multiplicative EKF** (Fehlerzustand als kleiner 3D-Rotationsvektor im Tangentialraum, multiplikativ aufs Quaternion angewandt).

2.5 Anwendung Tischtennisball (konstante Beschleunigung)

Zustand $\mathbf{x} = [x, y, z, \dot{x}, \dot{y}, \dot{z}]^\top$, Schwerkraft als bekannte Steuerung. Mit Zeitschritt Δt :

$$\mathbf{F} = \begin{pmatrix} \mathbf{I}_3 & \Delta t \mathbf{I}_3 \\ \mathbf{0} & \mathbf{I}_3 \end{pmatrix}, \quad \mathbf{B}\mathbf{u} = \begin{pmatrix} \frac{1}{2}\Delta t^2 \mathbf{g} \\ \Delta t \mathbf{g} \end{pmatrix}, \quad \mathbf{g} = \begin{pmatrix} 0 \\ 0 \\ -9.81 \end{pmatrix}, \quad \mathbf{H} = \begin{pmatrix} \mathbf{I}_3 & \mathbf{0} \end{pmatrix} \quad (2.4)$$

Die Stereokamera liefert nur die Position (daher $\mathbf{H}$ projiziert auf die ersten drei Komponenten); der Filter schätzt die Geschwindigkeit mit und extrapoliert per Prädiktion (ohne Messupdate) den Flug bis zur Schlagebene – das ist die Vorhersage des Abschlagzeitpunkts. Luftwiderstand und Magnus-Effekt (Spin) machen das Modell nichtlinear $\rightarrow$ EKF, falls nötig.

Kapitel 3

Odometrie und Sensorfusion

Odometrie schätzt die Eigenbewegung aus inkrementellen Sensordaten.

- **Radodometrie:** Aus Encoder-Ticks und Differentialantriebs-Kinematik die Pose integrieren. Einfach, aber driftet (Schlupf, Rundungsfehler akkumulieren).
- **Visuelle Odometrie (VO):** Bewegung aus der Verfolgung von Bildmerkmalen über aufeinanderfolgende Frames schätzen. Mit Stereokamera bekommst du den Maßstab gratis (Tiefe ist bekannt).
- **Visual-Inertial Odometry (VIO):** VO + IMU per EKF fusioniert – robuster, glatter.

In ROS2 ist `robot_localization` das Standardpaket: ein `ekf_node`/`ukf_node` fusioniert mehrere Quellen (Rad-Odom + IMU + visuelle Odom) zu einer konsistenten `odom`→`base_link`-Transformation. Drift wird später durch SLAM/Loop-Closure korrigiert (`map`→`odom`).

Kapitel 4

Stereosehen und Merkmalerkennung

4.1 Lochkameramodell und Tiefe aus Disparität

Eine kalibrierte Kamera bildet einen 3D-Punkt über die Intrinsik-Matrix $\mathbf{K}$ ab:

$$\mathbf{K} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \quad (4.1)$$

Bei zwei horizontal um die **Basislinie** b versetzten Kameras erscheint ein Punkt im linken und rechten Bild an leicht verschiedener Spalte; die Differenz ist die **Disparität** d . Die Tiefe folgt direkt:

$$Z = \frac{f \cdot b}{d} \quad (4.2)$$

Große Basislinie $\rightarrow$ bessere Tiefenauflösung in der Ferne, aber kleinerer überlappender Sichtbereich in der Nähe. Die Disparität wird auf *rektifizierten* Bildern berechnet (beide Bildebenen koplanar, Epipolarlinien horizontal), typisch mit `StereoBM` (schnell) oder `StereoSGBM` (genauer) aus OpenCV.

4.2 Kalibrierung

Beide Kameras müssen kalibriert werden (Intrinsik + Verzerrung) und zueinander (Extrinsik: $\mathbf{R}, \mathbf{t}$ zwischen den Linsen). Standard ist ein Schachbrettmuster und das ROS 2-Paket `camera_calibration` aus dem `image_pipeline`. Ergebnis sind `.yaml`-Dateien, die der Treiber lädt. **Ohne saubere Kalibrierung ist jede Tiefenmessung wertlos** – das ist der erste reale Stolperstein gegenüber der Simulation, wo die “Kamera” perfekt kalibriert ist.

4.3 Merkmalerkennung ohne eigenes KI-Training

Es gibt ausgereifte *klassische* Detektoren, die ohne Trainingsdaten und ohne neuronale Netze auskommen. Sie finden markante, wiedererkennbare Bildpunkte (Ecken, Blobs) und beschreiben deren lokale Umgebung:

Detektor	Eigenschaft	Einsatz
ORB	schnell, binär, lizenzfrei	Echtzeit-Matching, VO, Standardwahl für Hobby
SIFT	skalierungs-/rotationsinvariant, robust	genaues Matching, CAD-Posenschätzung
AKAZE	gute Balance	Alternative zu SIFT/ORB

BRISK	binär, schnell	ressourcenarme Plattformen
--------------	----------------	----------------------------

Alle sind in OpenCV enthalten (`cv2.OORB_create()` usw.) und über `cv_bridge` in ROS2 nutzbar. Das Matching erfolgt mit `BFMatcher` (Brute Force) oder `FLANN`, gefolgt von Ausreißerfilterung (Lowe-Ratio-Test, RANSAC). Für **Bewegungserkennung** (Projekt 3) brauchst du nicht einmal Merkmale: Hintergrundsubtraktion (`cv2.createBackgroundSubtractorMOG2`) oder optischer Fluss (`calcOpticalFlowFarneback`) reichen.

4.4 Posenschätzung – 6D-Lage eines Bauteils

Für Projekt 4 (CAD-Matching) gibt es zwei Hauptwege:

1. **2D-3D über PnP:** Merkmale im Kamerabild gegen bekannte 3D-Punkte des CAD-Modells matchen, dann `cv2.solvePnP` liefert Rotation + Translation des Bauteils relativ zur Kamera.
2. **3D-3D über ICP:** Aus der Stereokamera eine Punktwolke erzeugen und per **Iterative Closest Point** an die aus dem CAD-Modell gesampelte Punktwolke anpassen (z. B. mit Open3D). Liefert die volle 6D-Pose, robust bei guter Initialschätzung.

Das Ergebnis (Rotation als Quaternion + Translation) wird in TF veröffentlicht und vom Greifplaner (MoveIt 2) konsumiert.

Kapitel 5

6-DoF-Posenschätzung mit gelernten Surface Embeddings

Dieses Kapitel vertieft die theoretischen Grundlagen hinter Projekt 4. Es baut direkt auf dem Stereosehen (Kapitel zuvor) auf: Dort wurde die 6D-Pose über klassische Merkmale (PnP, ICP) bestimmt; hier wird der moderne, lernbasierte Weg über **dichte Surface Embeddings** (in der Linie von **SurfEmb** [1]) entwickelt, der bei texturarmen, glänzenden oder symmetrischen Industrieteilen robuster ist.

5.1 Das Problem: sechs Freiheitsgrade

6-DoF-Posenschätzung bestimmt *Position und Orientierung* eines bekannten Bauteils – drei Translations- und drei Rotationsfreiheitsgrade. **Warum überhaupt?** Ein Werkstück liegt selten zweimal gleich (Schüttgut in Kisten, verschobene Vorrichtung, Förderband). Der Roboter muss das Teil *sehen*, bevor er greift. Gesucht ist die starre Transformation, die das bekannte CAD-Modell in das Kamera-/Roboterkoordinatensystem überführt:

$$\boxed{\mathbf{p}_{\text{cam}} = \mathbf{R} \mathbf{p}_{\text{model}} + \mathbf{t}} \quad \mathbf{R} \in SO(3), \quad \mathbf{t} \in \mathbb{R}^3 \quad (5.1)$$

Die rücktransformierte (“ausgerichtete”) Punktwolke ist nur die visuelle Bestätigung – das eigentliche Ergebnis ist $(\mathbf{R}, \mathbf{t})$.

5.2 Rotationsrepräsentationen

Die Translation ist trivial ($\mathbb{R}^3$); die Rotation ist der heikle Teil, weil $SO(3)$ gekrümmt ist. Gängige Darstellungen mit ihren Fallstricken:

Darstellung	Eigenschaft / Fallstrick
Rotationsmatrix (3×3)	9 Zahlen, 6 Nebenbedingungen (orthonormal); redundant aber stabil
Euler-Winkel	nur 3 Zahlen, aber Gimbal Lock (Freiheitsgrad-Verlust)
Quaternion	4 Zahlen, singularitätsfrei, aber Double Cover ($\mathbf{q}$ und $-\mathbf{q}$ = gleiche Drehung)
Kontinuierliche 6D-Form	beste Wahl für die NN-Regression – stetig, keine Sprünge [2]

Für neuronale Netze ist die Stetigkeit entscheidend: Euler und Quaternion haben Unstetigkeiten,

an denen das Netz schlecht lernt. Die 6D-Darstellung (zwei Spaltenvektoren $\rightarrow$ Gram-Schmidt $\rightarrow$ $\mathbf{R}$) ist überall stetig.

5.3 Warum gelernte Embeddings statt Regression

Zwei naive Wege scheitern in der Industrie:

- **Handgefertigte Deskriptoren** (FPFH [3], SIFT) versagen bei texturlosen, glänzenden, symmetrischen Teilen und unter Verdeckung – genau die typischen Fälle in der Fertigung.
- **Direkte Pose-Regression** (Netz $\rightarrow$ ein $(\mathbf{R}, \mathbf{t})$) nimmt *eine* Antwort an (unimodal) und **bricht bei Symmetrie**: ein symmetrisches Teil hat mehrere korrekte Posen, der Mittelwert ist falsch.

Gelernte dichte Surface Embeddings lösen beides: Ein Netz bildet jeden Oberflächenpunkt in einen Merkmalsraum ab; gemessene Punkte werden über die *Nähe im Merkmalsraum* den Punkten des bekannten CAD-Modells zugeordnet. Statt einer Punktschätzung lernt das Modell eine **Verteilung** über Korrespondenzen – Symmetrie wird implizit behandelt (mehrere Oberflächenpunkte teilen sich dann schlicht ein Embedding) [1].

5.4 Kontrastives Lernen der Embeddings

Trainiert werden die Embeddings mit einem **kontrastiven Verlust**: wahre Pixel- $\leftrightarrow$ -Punkt-Paare werden im Merkmalsraum zusammengezogen, alle anderen Paare auseinandergedrückt. Die *Positiv*-Paare sind die unter bekannter Render-Pose bekannten Korrespondenzen, die *Negativ*-Paare andere gesampelte Oberflächenpunkte. In InfoNCE-Form für ein Query-Embedding $\mathbf{q}$ mit positivem Schlüssel $\mathbf{k}^+$:

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(\mathbf{q}, \mathbf{k}^+)/\tau)}{\sum_j \exp(\text{sim}(\mathbf{q}, \mathbf{k}_j)/\tau)} \quad (5.2)$$

mit Kosinus-Ähnlichkeit $\text{sim}(\cdot)$ und **Temperatur** τ (Tuning-Größe). **Der Clou**: die Labels sind quasi gratis – man rendert das CAD-Modell in bekannten Posen, womit Ground-Truth-Pose *und* Korrespondenzen automatisch vorliegen; keine manuellen Symmetrie-Labels nötig.

5.5 Das Two-Tower-Modell (Query–Key)

Die Architektur besteht aus zwei getrennten Türmen, die erst im gemeinsamen Embedding-Raum zusammentreffen:

- **Query-Tower** – *Eingang*: Bildausschnitt (2D-Pixel, RGB/RGB-D); *Ausgang*: Embedding pro Pixel; *Architektur*: großes CNN-Encoder-Decoder-Netz (U-Net / ResNet).
- **Key-Tower** – *Eingang*: eine 3D-Oberflächenkoordinate (x, y, z) vom CAD; *Ausgang*: Embedding dieses Punktes im selben Raum; *Architektur*: kleines MLP (wenige Schichten reichen, da der Eingang simpel ist – die **Asymmetrie** der Türme ist gewollt).

Beide werden *gemeinsam* mit dem kontrastiven Verlust trainiert, sodass zusammengehörige Pixel- und Punkt-Embeddings nahe beieinander landen. Ein Turm spricht “2D-Bild”, der andere “3D-Oberfläche”; der gemeinsame Raum ist die gemeinsame Sprache.

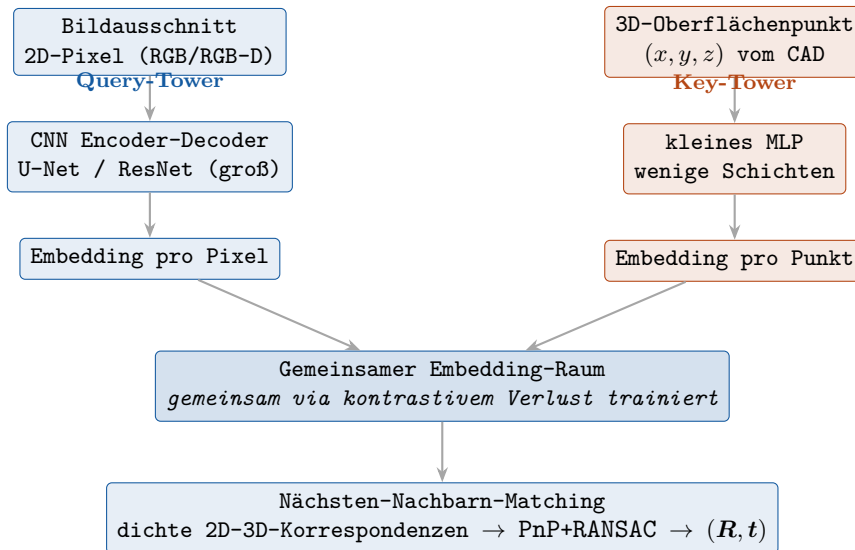


Abbildung 5.1: Two-Tower-Modell: die Türme bleiben getrennt bis zum gemeinsamen Embedding-Raum – genau diese Trennung *ist* die Architektur. Bei der Inferenz wird jedes Pixel-Embedding seinem nächsten Oberflächenpunkt-Embedding zugeordnet.

5.6 Von Korrespondenzen zur Pose: PnP + RANSAC

Aus den dichten 2D-3D-Korrespondenzen wird die Pose über **Perspective-n-Point** (PnP) gelöst – die $(\mathbf{R}, \mathbf{t})$, die die 3D-Modellpunkte $\mathbf{P}_i$ auf die beobachteten Pixel $\mathbf{u}_i$ projiziert:

$$(\mathbf{R}, \mathbf{t}) = \arg \min_{\mathbf{R}, \mathbf{t}} \sum_i \|\mathbf{u}_i - \pi(\mathbf{K}(\mathbf{R}\mathbf{P}_i + \mathbf{t}))\|^2 \quad (5.3)$$

mit Projektion $\pi(\cdot)$ und Intrinsik $\mathbf{K}$. **RANSAC** umhüllt das Ganze und verwirft Ausreißer-Korrespondenzen (zieht zufällige minimale Mengen, zählt Inlier, behält das beste Modell). Bei reinen 3D-3D-Daten tritt an die Stelle der Reprojektion die euklidische Punktdistanz.

5.7 Registrierung: grob und fein

Eine *einstufige* Registrierung ist unzuverlässig; Standard sind **zwei Stufen**:

1. **Grob (global):** liefert die “richtige Größenordnung” aus Merkmalen – Deskriptoren auf Quelle und Modell matchen, grobes $(\mathbf{R}, \mathbf{t})$ lösen (z. B. SAC-IA + FPFH). Braucht *keine* gute Startschätzung, ist aber nur näherungsweise.
2. **Fein = ICP** (*Iterative Closest Point* [4]): zu jedem Quellpunkt den nächsten Zielpunkt suchen, die Gesamtdistanz minimieren, wiederholen bis zur Konvergenz:

$$E(\mathbf{R}, \mathbf{t}) = \sum_i \|\mathbf{R}\mathbf{p}_i + \mathbf{t} - \mathbf{q}_{c(i)}\|^2, \quad c(i) = \text{nächster Nachbar} \quad (5.4)$$

Genau, aber nur **lokal**.

ICP-Schwächen: (1) *lokale Minima* – es läuft bergab und bleibt stecken; (2) *initialisierungsabhängig* – es funktioniert nur bei nahem Start (daher läuft die Grobstufe zuerst). **Go-ICP** [5] garantiert das globale Optimum, indem es ganz $SE(3)$ (alle starren Transformationen) per *Branch-and-Bound* durchsucht: Raum aufteilen, Fehler je Region beschränken, Regionen verwerfen, die nicht gewinnen können – langsamer, aber global optimal.

5.8 Punktwolken-Netze: PointNet und PointNet++

Verarbeitet man die Punktwolke direkt mit einem Netz, ist die zentrale Schwierigkeit: eine Punktwolke ist eine **ungeordnete Menge** – der Ausgang darf nicht von der Reihenfolge abhängen (**Permutationsinvarianz**).

- **PointNet** [6]: dasselbe geteilte MLP auf jeden Punkt, dann **Max-Pooling** (symmetrisch = reihenfolgeunabhängig). Erfasst die *globale* Form, verpasst aber lokale Details.
- **PointNet++** [7]: wendet PointNet *lokal* in Stufen an – Zentren sampeln (Farthest-Point-Sampling), Nachbarschaften gruppieren (Multi-Scale-Radien), lokale Merkmale extrahieren, auf größerer Skala wiederholen. Baut lokal + global hierarchisch auf, ähnlich einem CNN.

5.9 Stereo, Tiefe und Skalierung

Warum Stereo? Eine Mono-Kamera hat *Skalenmehrdeutigkeit* (klein-nah $\approx$ groß-fern). Stereo liefert Tiefe und löst die Skala – über Disparität, Basislinie und Brennweite (Triangulation, vgl. Stereo-Kapitel). Zwei Wege in die Pipeline:

- Tiefe als **Zusatzkanal** = RGB-D-Eingang, oder
- Tiefe + Intrinsik **rückprojizieren** $\rightarrow$ Punktwolke $\rightarrow$ 3D-3D-Registrierung.

Vorverarbeitung (vor dem Netz, ja): Kalibrierung (Intrinsik + Extrinsik) und Rektifizierung, Tiefen-Entrauschung, Umgang mit ungültiger Tiefe (glänzendes Metall liefert schlechte Tiefenwerte). **Auszahlung:** die Tiefe gibt *metrische* Skala – $(\mathbf{R}, t)$ in echten Millimetern, mit denen der Roboter handeln kann.

5.10 Zeitliche Konsistenz: Kalman-Pose-Tracking

Eine bildweise geschätzte Pose ist verrauscht/zittrig. Ein **Kalman-Filter** ergänzt *zeitliche Konsistenz* (vgl. Kapitel zu Kalman/EKF): der *Zustand* ist die Pose (ggf. plus Geschwindigkeit), die *Messung* ist die bildweise Netz-Pose. Prädiktion der nächsten Pose $\rightarrow$ Korrektur mit der neuen Schätzung. Das **glättet Jitter, verwirft Ausreißer-Frames und überbrückt kurze Verdeckung / ausgefallene Detektionen** – aus Einzelschuss wird *Tracking*, relevant für bewegtes Förderband oder armmontierte Kamera. Da Rotationen nichtlinear sind, nutzt man EKF/UKF oder filtert auf einer geeigneten Rotationsdarstellung.

5.11 Sim-to-Real und Trainingsdaten

Da die Labels durch Rendern des CAD-Modells praktisch gratis entstehen, wird synthetisch trainiert – aber es klappt ein **Sim-to-Real-Gap** (synthetisch $\rightarrow$ real: Rauschen, Beleuchtung, Reflektanz). Gegenmittel: **Domain Randomization**, physikalisch-basiertes Rendering (PBR, z. B. via Isaac Sim, vgl. Kapitel zu Simulationswerkzeugen) und ein kleiner realer Feintuning-Datensatz.

5.12 Benchmarking und Metriken

Größe	Bedeutung
ADD / ADD-S	mittlerer Punktabstand Modell $\leftrightarrow$ Schätzung; ADD-S nutzt nächsten Punkt (für <i>symmetrische</i> Teile)
Rotations-/Translationsfehler	geodätisch (Rotation) bzw. euklidisch (Translation), dazu RM-SE
BOP-Metriken	VSD, MSSD, MSPD $\rightarrow$ Average Recall [8]
Industrie-KPIs	Taktzeit/Latenz, Pick-Erfolgsrate, Falsch-Positiv-Rate

Datensätze	BOP-Suite, speziell T-LESS und ITODD (texturlos/industriell)
------------	--

Kernaussage in einem Satz

6-DoF-Posenschätzung über gelernte Surface Embeddings ersetzt fragile handgefertigte Deskriptoren durch ein *Zwei-Turm-Netz*, das Pixel und CAD-Oberflächenpunkte in einen gemeinsamen Merkmalsraum abbildet; das dichte Korrespondenz-Matching liefert über **PnP+RANSAC** (optional ICP-verfeinert) die starre Transformation $(\mathbf{R}, \mathbf{t})$ – robust auch bei texturlosen, glänzenden und symmetrischen Teilen.

Kapitel 6

Kinematik, Dynamik und Trajektorien

6.1 Vorwärtskinematik

Aus den Gelenkwinkeln $\mathbf{q}$ die Endeffektor-Pose berechnen. Klassisch über die Kette homogener 4×4 -Transformationen (Denavit-Hartenberg-Parameter). Der Rotationsblock jeder Transformation ist eine 3×3 -Matrix; intern äquivalent zur Quaternion-Darstellung.

6.2 Inverse Kinematik

Aus der gewünschten Pose die Gelenkwinkel finden. Numerisch über die **Jacobi-Matrix** $\mathbf{J}$, die Gelenkgeschwindigkeiten auf Endeffektorgeschwindigkeiten abbildet ($\dot{\mathbf{x}} = \mathbf{J}\dot{\mathbf{q}}$). Iterativ:

$$\Delta \mathbf{q} = \mathbf{J}^\dagger \mathbf{e}, \quad \mathbf{e} = \begin{pmatrix} \mathbf{e}_{\text{pos}} \\ \mathbf{e}_{\text{ori}} \end{pmatrix} \quad (6.1)$$

mit Pseudo-Inverser $\mathbf{J}^\dagger$ (oder Damped-Least-Squares nahe Singularitäten). Der **Orientierungsfehler** $\mathbf{e}_{\text{ori}}$ wird als Vektorteil des Fehlerquaternions $q_{\text{soll}} \otimes q_{\text{ist}}^{-1}$ gebildet – Euler-Differenzen sind hier wegen Gimbal Lock unbrauchbar. In ROS 2 löst **MoveIt 2** (mit KDL/TracIK) das für dich.

6.3 Dynamik

Das Bewegungsgesetz eines Roboterarms:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \boldsymbol{\tau} \quad (6.2)$$

$\mathbf{M}$ = Massenmatrix, $\mathbf{C}$ = Coriolis/Zentrifugal, $\mathbf{g}$ = Gravitation, $\boldsymbol{\tau}$ = Gelenkmomente. Für die Simulation ist entscheidend, dass du jedem Link in der URDF korrekte **Masse und Trägheitstensoren** gibst – sonst rechnet die Physik-Engine Unsinn.

6.4 Trajektorienplanung

- **Gelenkraum:** Kubische/quintische Polynome oder ein **trapezförmiges Geschwindigkeitsprofil** (beschleunigen – konstant – abbremesen) erzeugen ruckarme, ableitungsstetige Verläufe.
- **Kartesischer Raum:** Position linear interpolieren, Orientierung per **SLERP** (gleichmäßige Winkelgeschwindigkeit). Für glatte Mehrpunktbahnen **SQUAD**.

In ROS 2 transportiert `trajectory_msgs/JointTrajectory` eine Bahn (Punkte mit Position/Geschwindigkeit/Zeit) und der `joint_trajectory_controller` fährt sie ab.

Kapitel 7

Schrittmotoren und Drehmomentschätzung

7.1 NEMA-17-Grundlagen

Ein NEMA-17 ist ein bipolarer Hybridschrittmotor mit typisch $1,8^\circ/\text{Schritt} = 200$ Vollschritten/Umdrehung. Per **Microstepping** (z. B. 1/16, 1/256 beim TMC) wird sehr feinaufgelöst und vibrationsarm gefahren. Phasenströme typisch 0,4–2 A, Haltemoment je nach Baulänge $\approx 0,2$ –0,6 Nm. Im **offenen Regelkreis** kennt der Motor seine Last nicht – bei Überlast verliert er Schritte unbemerkt.

7.2 Indirekte Drehmomentschätzung

Statt eines teuren Drehmomentsensors wird das Lastmoment aus den *elektrischen* Größen geschätzt. Physikalischer Hintergrund: Bei einem belasteten Schrittmotor wächst der **Lastwinkel** (die Verschiebung zwischen Statorfeld und Rotor). Dieser zeigt sich in Phasenströmen und Gegen-EMK. Zwei praktische Umsetzungen:

1. **StallGuard (Trinamic):** Der Treiber misst intern die Gegen-EMK und liefert einen lastproportionalen Zahlenwert (SG_RESULT) über UART. Das ist der bevorzugte Ansatz – ein Proxy fürs Drehmoment ohne Zusatzhardware.
2. **Externe Strom-/Spannungsmessung:** Ein Sensormodul (z. B. INA226) misst Busspannung und -strom pro Treiber. Zusammen mit dem bekannten Microstep-Zustand lässt sich die mechanische Leistung und damit das Moment abschätzen.

Beide Wege sind weniger präzise als ein echter Sensor, aber günstig und praktikabel – siehe Teil II für die konkrete Hardware.

Kapitel 8

Zykloidgetriebe

Ein Zykloidgetriebe erzeugt auf kompaktem Bauraum eine hohe Übersetzung bei geringem Spiel (Backlash) und hoher Stoßfestigkeit – ideal als “letzte Stufe” vor dem Robotergelenk. Eine exzentrisch gelagerte Zykloidscheibe mit n Nocken wälzt in einem Ring mit N Bolzen ab. Die Übersetzung ist

$$i = \frac{N}{N - n}. \quad (8.1)$$

Mit einer Nockendifferenz von eins ($N - n = 1$) folgt $i = N$: 11 Ringbolzen und eine 10-Nocken-Scheibe ergeben 11:1.

Für die Simulation ist die innere Mechanik irrelevant – das Getriebe wird als *ein* angetriebenes Gelenk mit Übersetzung modelliert (siehe Teil IV, `ros2_control-Transmission`). Wichtig für die reale Auslegung: das übertragbare Ausgangsmoment, die Steifigkeit und das Restspiel, die du später in die Gelenklimits einträgst.

Teil II

Hardware-Auswahl

Kapitel 9

Komponenten im günstigen Hobby-Bereich

Leitprinzip

Fokus liegt nicht auf Elektrotechnik: Wo möglich Module und Plug-and-Play, niedrige Kosten. Die Architektur trennt sauber eine **Echtzeit-Motorebene** (Mikrocontroller, nah an den Treibern) von einer **High-Level-Ebene** (PC/Pi mit vollem ROS 2 für Trajektorie, Vision, Kalman).

9.1 Schrittmotor und Treiber

Komponente	Empfehlung	Begründung
Motor	NEMA-17 (z. B. 17HS19)	Standard, günstig, ausreichend Moment mit Getriebe
Treiber	TMC2209 (UART)	StallGuard4 liefert Lastwert über UART = indirekte Drehmomentschätzung; leise, Stepstick-Format, $\approx 5\text{--}10\text{€}$
Treiber-Alt. Strom/Spannung	TMC2130/5160 (SPI) INA226 -Modul (I ² C)	höherer Strom / SPI statt UART misst Busspannung + Strom direkt, plug-and-play, $\approx 2\text{€}$ pro Kanal
Closed-Loop (opt.)	MKS SERVO42 / BTT S42B	integrierter Encoder + Treiber, falls echter Regelkreis gewünscht

Konkret für den Regelungsansatz: Der TMC2209 im UART-Modus gibt `SG_RESULT` (Last-proxy) ohne Zusatzhardware. Willst du *zusätzlich* Busspannung und -strom explizit messen, setzt du je Achse ein INA226-Modul auf die Treiberversorgung. Beide hängen am selben I²C/UART-Bus des Mikrocontrollers.

9.2 Recheneinheiten

ESP32 (Motorebene)

Dual-Core, viele PWM/GPIO, FreeRTOS, harte Echtzeit für Schrittgenerierung und das Auslesen von TMC (UART) und INA226 (I²C). Entscheidend: **micro-ROS** bindet den ESP32 als vollwertigen ROS 2-Knoten in den Graphen ein.

Raspberry Pi (High-Level)

Pi 4/5 mit Ubuntu kann den vollen ROS 2-Stack tragen (Trajektorienplanung, Kalman). Der **Pi**

Zero ist dafür zu schwach – für Vision ungeeignet; nutze ihn höchstens als micro-ROS-Brücke. Vision und schweres Rechnen gehören auf den externen PC.

Empfohlene Arbeitsteilung

Externer PC (Ubuntu + ROS 2): Trajektorienplanung, Stereo-Vision, Kalman-Filter, Visualisierung. $\longleftrightarrow$ (WLAN/USB) $\longleftrightarrow$ **ESP32** (micro-ROS): empfängt Sollwerte, generiert Schritte, liest StallGuard/INA226, publiziert Gelenkzustände. Die Stereokamera hängt per USB am PC.

9.3 Stereokamera

Das ELP-Modell ist eine synchronisierte Doppelobjektiv-USB-Kamera (UVC-konform). Sie meldet sich unter Linux als V4L2-Gerät (`/dev/video*`) und liefert in der Regel ein *nebeneinanderliegendes* (side-by-side) Bild über eine USB-Verbindung – die Synchronisierung beider Sensoren ist der Vorteil gegenüber zwei Einzelkameras (wichtig für bewegte Objekte wie den Ball). In ROS 2:

- Treiber: `v4l2_camera` oder `usb_cam`.
- Side-by-side-Frame in linkes/rechtes Bild splitten (kleiner eigener Knoten, siehe Teil VI).
- Eigene Stereokalibrierung ist Pflicht – das Modul ist nicht vorkalibriert.

Kapitel 10

Optimierungspotenzial für ein eigenes PCB

Diese Punkte sind *Erweiterung*, nicht Voraussetzung – mit Modulen kommst du zunächst aus. Für ein späteres eigenes Board:

- **Treiber integrieren** statt Stepstick-Module: TMC2209 direkt bestücken – spart Bauhöhe, bessere Thermik über Kupferflächen und Thermovias unter dem Chip.
- **Strom-/Spannungsmessung onboard:** INA-Sensor mit dimensioniertem Shunt je Kanal nahe am Treiber, kurze Kelvin-Anbindung des Shunts.
- **Power-Integrity:** Bulk-Elko plus keramische Stützkondensatoren direkt an jedem Treiber; getrennte Power-/Logik-Masse mit Sternpunkt (Single-Point-Ground).
- **Signalführung:** STEP/DIR bzw. UART als kurze, von der Leistung getrennte Leitungen; bei gemischten Pegeln (3,3 V/5 V) Levelshifter vorsehen.
- **Schutz:** ESD-Dioden an USB- und Encoder-Leitungen; Verpolschutz; Freilauf-/EMV-Betrachtung der Motorleitungen.
- **Konnektivität:** JST-Stecker für Motoren/Encoder, Schraubklemme für die Versorgung; optional CAN-Transceiver für saubere Mehrachs-Verkettung statt vieler UARTs.
- **Onboard-Encoder-Interface** für späteren Closed-Loop-Betrieb (FOC).
- **Thermik:** Thermopad/Kühlfläche unter den Treibern, definierter Luftstrom.

Teil III

Software-Setup: VM, Linux, ROS 2

Kapitel 11

Betriebssystem und Distribution

11.1 Distributionswahl (Stand Juni 2026)

ROS 2-Releases erscheinen jährlich; relevant sind die LTS-Versionen. Für ein langfristiges Lernprojekt:

Distribution	Ubuntu	Support bis	Eignung
Jazzy Jalisco	24.04	2029 (LTS)	Empfehlung: reif, viele Tutorials, von Isaac Sim unterstützt
Humble Hawksbill	22.04	2027 (LTS)	meiste Tutorials, etwas älter
Kilted Kaiju	24.04	~2026	non-LTS, kurzer Support – meiden
Lyrical Luth	24.04	2031 (LTS)	brandneu (Mai 2026), Ökosystem noch nicht nachgezogen

Empfehlung

Ubuntu 24.04 LTS + ROS 2 Jazzy Jalisco. LTS bis 2029, ausgereifte Tutorials, von Gazebo Harmonic und NVIDIA Isaac Sim unterstützt. Lyrical Luth ist erst ~10 Tage alt – für den Einstieg zu frisch.

11.2 VM, Dual-Boot oder WSL2?

Wichtige Einschränkung bei der VM

Eine VM (VirtualBox/VMware) ist für die *ROS 2-Grundlagen* in Ordnung, aber **Gazebo Harmonic und Isaac Sim brauchen GPU-Beschleunigung**, die in VMs schlecht durchgereicht wird – 3D-Simulation läuft dort zäh. Optionen, nach Eignung:

1. **Natives Ubuntu (Dual-Boot)** – beste Leistung, empfohlen sobald du simulierst.
2. **WSL2 unter Windows** – inzwischen mit GPU-Zugriff (WSLg), guter Kompromiss.
3. **VM** – nur für CLI/Tutorials; für schwere Simulation ungeeignet.

Kapitel 12

ROS 2 installieren – Schritt für Schritt

```

1  # 1) Locale auf UTF-8
2  sudo apt update && sudo apt install -y locales
3  sudo locale-gen en_US en_US.UTF-8
4  sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
5
6  # 2) Universe-Repo + Schlüssel + ROS2-Quelle
7  sudo apt install -y software-properties-common curl
8  sudo add-apt-repository universe
9  sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
10     -o /usr/share/keyrings/ros-archive-keyring.gpg
11  echo "deb [arch=$(dpkg --print-architecture) \
12     signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
13     http://packages.ros.org/ros2/ubuntu \
14     $(. /etc/os-release && echo $UBUNTU_CODENAME) main" \
15     | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
16  sudo apt update

```

Listing 12.1: Locale + ROS2-Apt-Quelle (Ubuntu 24.04)

```

1  # 3) ROS2 Desktop (inkl. RViz2, Demos)
2  sudo apt install -y ros-jazzy-desktop
3
4  # 4) Build-Werkzeuge
5  sudo apt install -y ros-dev-tools \
6     python3-colcon-common-extensions python3-rosdep
7  sudo rosdep init && rosdep update
8
9  # 5) In jeder Shell verfuegbar machen
10  echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc && source ~/.bashrc
11
12  # 6) Funktionstest (zwei Terminals)
13  ros2 run demo_nodes_cpp talker          # Terminal A
14  ros2 run demo_nodes_py listener         # Terminal B

```

Listing 12.2: ROS2 Jazzy + Entwicklungswerkzeuge

12.1 Pakete, die du für die Projekte brauchst

Paket (apt: ros-jazzy-...)	Zweck
----------------------------	-------

<code>ros-gz</code> (Gazebo Harmonic Bridge)	Simulation ↔ ROS 2
<code>ros2-control</code> , <code>ros2-controllers</code>	Hardware-Abstraktion, <code>joint_trajectory_controller</code>
<code>gz-ros2-control</code>	<code>ros2_control</code> -Plugin für Gazebo
<code>moveit</code>	Bewegungsplanung, IK, Greifen (Projekt 4)
<code>navigation2</code> , <code>nav2-bringup</code>	Autonome Navigation (Projekt 3)
<code>slam-toolbox</code>	SLAM/Kartierung (Projekt 3)
<code>robot-localization</code>	EKF/UKF-Sensorfusion (Odometrie)
<code>image-pipeline</code>	<code>camera_calibration</code> , <code>stereo_image_proc</code>
<code>v4l2-camera</code> bzw. <code>usb-cam</code>	Treiber für die USB-Stereokamera
<code>vision-opencv</code> (<code>cv_bridge</code>)	OpenCV ↔ ROS 2-Bildnachrichten
<code>xacro</code> , <code>robot-state-publisher</code>	URDF/Xacro, TF-Veröffentlichung
<code>rqt</code> , <code>rqt-graph</code> , <code>rqt-image-view</code>	Introspektion/Visualisierung

Installation gebündelt, z. B. `sudo apt install ros-jazzy-ros-gz ros-jazzy-ros2-control ros-jazzy-moveit ...`. Für den ESP32 zusätzlich der **micro-ROS Agent** (separat aus dem micro-ROS-Workspace gebaut).

Teil IV

Simulation und der Weg vom CAD-Modell zur Realität

Kapitel 13

Simulationswerkzeuge im ROS 2-Ökosystem

Gazebo Classic ist seit **Januar 2025 EOL**; “modernes Gazebo” (früher Ignition) ist der Nachfolger:

Werkzeug	Charakter	Eignung
Gazebo Harmonic	Standard-Simulator für Jazzy, Bridge <code>ros_gz</code>	Allrounder: Physik, Sensoren (Kamera, Tiefe, IMU, LiDAR), <code>gz_ros2_control</code> . Erste Wahl.
NVIDIA Isaac Sim	photorealistisch, GPU	Vision/Pose-Schätzung mit realistischen Bildern; braucht starke GPU; nur LTS-Distros
Webots	einfach, didaktisch	schneller Einstieg, gute ROS 2-Anbindung
MuJoCo	schnelle, präzise Dynamik	kontaktreiche Regelung, RL; weniger Sensor-Realismus
PyBullet	leichtgewichtig	schnelles Prototyping von Kinematik/Dynamik

RViz2 ist kein Simulator, sondern Visualisierung – du nutzt es parallel.

13.1 Was lässt sich vorab simulieren?

Sehr viel – die meiste *Logik* der vier Projekte ist simulierbar:

- **Kinematik/Dynamik** des Stepper-Roboters: Gelenkbewegung, Trajektorienabfahrt, Kollisionen, Massen/Trägheiten.
- **Sensoren:** simulierte Stereokamera (gerenderte Bilder – du kannst deine Merkmalerkennung darauf testen), Tiefenbild, IMU, LiDAR, Odometrie.
- **Tischtennisball:** Projekttilflug inkl. Gravitation und Aufprall als Starrkörperphysik; dein Kalman-Filter kann auf den simulierten 3D-Positionen laufen.
- **Navigation:** Nav2 + simulierte Odometrie/LiDAR in einer Welt.
- **Greifen:** MoveIt 2-Planung in simulierter Szene mit platzierten Bauteilen.

Kannst du die Projekte vollständig in Simulation umsetzen?

Die Steuerungs-, Planungs- und Wahrnehmungs-Logik: ja. Du kannst Trajektorienregelung, Kalman-Schätzung, Navigation und die Vision-Pipeline komplett in Gazebo entwickeln und testen. **Was die Simulation nicht abbildet:** reales Kameraräuschen/Kalibrierfehler, Schrittverluste und Resonanzen des realen Steppers, exakte Kontaktdynamik (Ballspin/-absprung sind Näherungen), reale Latenzen und Zeitverhalten. Diese Effekte treten erst auf der echten Hardware zutage.

13.2 MuJoCo und Isaac Sim im Detail

Zwei Werkzeuge der obigen Tabelle verdienen eine genauere Betrachtung, weil sie in der modernen Robotik – besonders im Lernen von Regelungen (*Reinforcement Learning*, RL) und in der Wahrnehmung – eine Sonderstellung einnehmen. Sie lösen Probleme, an denen Gazebo als Allzweck-Simulator an seine Grenzen stößt: **MuJoCo** bei der schnellen, präzisen Kontaktdynamik, **Isaac Sim** beim photorealistischen Rendern und beim massiv parallelen GPU-Training.

13.2.1 MuJoCo – Multi-Joint dynamics with Contact

MuJoCo (*Multi-Joint dynamics with Contact*) ist eine quelloffene Physik-Engine, ursprünglich 2012 von Emanuel Todorov vorgestellt [9] und seit 2021 von Google DeepMind entwickelt und unter Apache-2.0 frei verfügbar [10]. Ihr Markenzeichen ist ein **stabiler, weicher Kontaktlöser** (konvexe Optimierung pro Zeitschritt), der kontaktreiche Vorgänge – Greifen, Laufen, Anschlagen – numerisch stabil und schnell rechnet.

- **Stärken:** sehr schnelle und genaue Dynamik (oft schneller als Echtzeit, hunderte bis tausende Schritte/s), differenzierbare Physik, analytisch saubere Gelenk-/Sehnenmodelle, exzellent für Regelungs- und RL-Forschung. Mit **MJX** läuft MuJoCo über JAX auf der GPU und simuliert tausende Welten parallel.
- **Schwächen:** *kein* photorealistisches Rendering (einfacher OpenGL-Viewer), weniger fertige Sensor-Plugins als Gazebo, Modelle in eigenem MJCF-XML (URDF-Import möglich, aber oft mit Nacharbeit).
- **Robotik-Relevanz:** De-facto-Standard im RL-Benchmarking (Gymnasium/dm_control), Lokomotion vierbeiniger Roboter, geschickte Manipulation, Modellprädiktive Regelung (MPC). Wenn die *Kontaktphysik* und die *Trainingsgeschwindigkeit* zählen, ist MuJoCo erste Wahl.

13.2.2 NVIDIA Isaac Sim – photorealistisch und GPU-parallel

Isaac Sim ist NVIDIAS Robotik-Simulator auf Basis der **Omniverse**-Plattform und des USD-Szenenformats [11]. Die Physik liefert **PhysX 5** (GPU-beschleunigt), das Rendering nutzt **RTX-Raytracing** – also physikalisch plausibles Licht, Schatten und Materialien.

- **Stärken:** *photorealistische* Kamerabilder, exzellente Erzeugung synthetischer Trainingsdaten (*Synthetic Data*, inkl. automatischer Labels: Segmentierung, Bounding-Boxes, Tiefe), *Domain Randomization* für robuste Vision-Modelle, GPU-parallele Physik für großskaliges RL über **Isaac Lab** [12].
- **Schwächen:** braucht eine starke NVIDIA-RTX-GPU (VRAM-hungrig), hohe Einstiegshürde, große Installation; weniger “leichtgewichtig” als Gazebo oder MuJoCo für schnelle Iteration.
- **Robotik-Relevanz:** überall dort, wo das *Aussehen* der Szene entscheidend ist – Training von Objekterkennung/Pose-Schätzung, Sim-to-Real für kamerabasierte Politiken, digitale Zwillinge ganzer Fertigungszellen.

13.3 Lassen sie sich integriert einsetzen?

Ja – aber meist arbeitsteilig, nicht als ein gemeinsamer Simulationskern. Beide sprechen ROS 2, sodass sie sich in dieselbe Knoten- und Topic-Architektur einfügen wie Gazebo. Der gemeinsame Nenner ist wieder `ros2_control` (vgl. das folgende Kapitel zur Sim-Real-Trennung): die High-Level-Logik bleibt identisch, nur die unterste Schicht – der Simulator hinter dem Hardware-Interface – wird getauscht.

Werkzeug	ROS 2-Anbindung	Typische Rolle im Workflow
Gazebo Harmonic	<code>ros_gz-Bridge</code> , <code>gz_ros2_control</code>	Allround-Integrationstest, Sensorik, Nav2/MoveIt2
MuJoCo	<code>mujoco_ros2_control</code> , eigene Bridges	Regler-/RL-Training, Kontaktdynamik, MPC
Isaac Sim	natives ROS 2-Bridge-Extension	Vision-Trainingsdaten, photorealistische Pose-Schätzung, RL via Isaac Lab

Der heute verbreitete **kombinierte Workflow** sieht so aus:

1. **Politik/Regler lernen** – massiv parallel in **Isaac Lab** (auf Isaac Sim) oder in **MuJoCo/MJX** auf der GPU. Hier zählen Geschwindigkeit und Kontaktphysik, nicht das Aussehen.
2. **Wahrnehmung trainieren** – photorealistische, automatisch gelabelte Bilder aus **Isaac Sim** für die Objekt-/Pose-Erkennung (Domain Randomization gegen den Sim-to-Real-Gap).
3. **Integrieren und gegentesten** – die fertigen Knoten im **Gazebo**-Allroundmodell oder direkt am realen Roboter über `ros2_control` verifizieren.

Einordnung für dieses Projekt

Für den Stepper- und Tischtennisroboter ist **Gazebo Harmonic die praktische Basis**: Kinematik, Sensorik, Kontakt Ball/Schläger, `ros2_control`. **MuJoCo** lohnt sich, falls die *Schlag-/Kontaktdynamik* oder eine *gelernte Schlagpolitik* (RL) genauer untersucht werden soll – es rechnet kontaktreiche Vorgänge schneller und stabiler. **Isaac Sim** wird relevant, wenn die *Ballerkennung* mit photorealistischen, gelabelten Bildern trainiert werden soll, statt sie nur an realen Kamerabildern abzustimmen. Beide setzen voraus: MuJoCo profitiert von einer GPU (MJX), Isaac Sim *erfordert* eine starke NVIDIA-RTX-GPU – das ist die wichtigste Hürde gegenüber dem leichten Gazebo-Setup.

Kapitel 14

Wo sich Simulation und reale Implementierung trennen

Das ist die architektonisch wichtigste Einsicht des ganzen Dokuments. Der Trick von **ros2_control**: deine High-Level-Knoten (Trajektorienplaner, Vision, Kalman, Nav2) bleiben *identisch*. Was sich ändert, ist nur die unterste Schicht – das **Hardware-Interface-Plugin**.

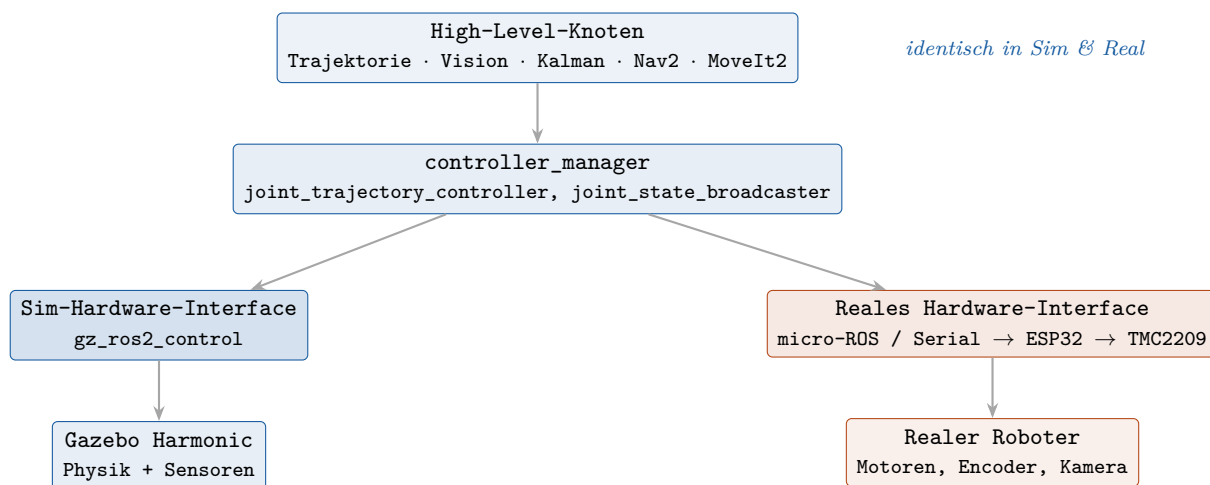


Abbildung 14.1: Sim-zu-Real: nur die unterste Schicht (Hardware-Interface) wird getauscht. Alles darüber bleibt gleich.

Konkret: In der Simulation lädt die URDF das Plugin `gz_ros2_control/GazeboSimSystem`. Für die echte Maschine schreibst du ein eigenes `hardware_interface::SystemInterface`, das Sollwerte an den ESP32 schickt (micro-ROS oder serielle Brücke) und Gelenkzustände (Encoder/StallGuard) zurückliest. **Dieselben Controller, dieselben Topics, dieselbe Trajektorie** – du tauschst eine Plugin-Zeile und die Hardware. Genauso bei der Kamera: simulierte Bildquelle vs. `v4l2_camera` – die nachgelagerte Vision-Pipeline merkt keinen Unterschied.

Kapitel 15

Vorabschritte: vom CAD-Modell zur Simulation

Bevor du simulierst, brauchst du ein **URDF** (Unified Robot Description Format) – die XML-Beschreibung deines Roboters aus *Links* (starre Körper) und *Joints* (Gelenke dazwischen).

15.1 Getrennte STL-Exporte pro Gelenk

Wenn zwischen Teil A und Teil B eine Gelenkverbindung besteht, müssen beide *separat* als Mesh (STL/DAE) exportiert werden – denn jedes ist ein eigener Link mit eigenem Koordinatensystem, und das Gelenk dazwischen wird in der URDF definiert, nicht im CAD-Mesh. Vorgehen:

1. Im CAD die Gelenkachsen und je Link ein Referenz-Frame festlegen (idealerweise Mesh-Ursprung = Gelenkpunkt – erspart später viel Frame-Frust).
2. **Jeden Link einzeln** als STL exportieren (Teil A, Teil B, ...), bevorzugt in seinem eigenen Frame.
3. Masse + Trägheitstensor je Link aus dem CAD übernehmen (für korrekte Dynamik).
4. URDF schreiben: Links mit `visual/collision/inertial`; Joints mit `parent/child, origin` (Lage des Gelenks), `axis` und `limit`.

Automatisierung – nicht von Hand schreiben

Nutze einen CAD-zu-URDF-Exporter statt das XML manuell zu tippen: **sw2urdf** (SolidWorks), **onshape-to-robot** (OnShape) oder Fusion-360-Plugins. Du wählst Gelenkachsen und Referenz-Frames interaktiv; das Tool exportiert die Meshes *und* eine konsistente URDF mit korrekten Frames und Trägheiten.

```

1 <link name="teilA">
2   <visual>
3     <geometry>
4       <mesh filename="package://my_robot/meshes/teilA.stl"/>
5     </geometry>
6   </visual>
7   <collision>
8     <geometry>
9       <mesh filename="package://my_robot/meshes/teilA.stl"/>
10    </geometry>
11  </collision>
12  <inertial>
13    <mass value="0.45"/>
14    <inertia ixx="0.001" iyy="0.001" izz="0.0008"
15              ixy="0" ixz="0" iyz="0"/>

```

```

16   </inertial>
17 </link>
18
19 <joint name="gelenk_AB" type="revolute">
20   <parent link="teilA"/>
21   <child link="teilB"/>
22   <origin xyz="0 0 0.08" rpy="0 0 0"/>
23   <axis xyz="0 0 1"/>
24   <limit lower="-1.57" upper="1.57" effort="12.0" velocity="3.0"/>
25 </joint>
26
27 <link name="teilB"> <!-- visual/collision/inertial wie oben --> </link>

```

Listing 15.1: URDF-Auszug: zwei Links mit Drehgelenk

15.2 Gelenktypen

- **revolute** – Drehgelenk mit Anschlag (Limits). Der Normalfall für Roboterarme.
- **continuous** – Drehgelenk ohne Anschlag (Räder).
- **prismatic** – Linearführung.
- **fixed** – starre Verbindung (z. B. Kamera an Basis).

15.3 Getriebeübersetzung in `ros2_control`

Motor + Zykloidgetriebe werden als *ein* angetriebenes Gelenk abstrahiert. Die Übersetzung steckt in der **`ros2_control-Transmission`**:

```

1 <transmission name="trans_gelenk_AB">
2   <type>transmission_interface/SimpleTransmission</type>
3   <joint name="gelenk_AB">
4     <hardwareInterface>
5       hardware_interface/PositionJointInterface
6     </hardwareInterface>
7   </joint>
8   <actuator name="motor_AB">
9     <mechanicalReduction>40</mechanicalReduction> <!-- 40:1 -->
10  </actuator>
11 </transmission>

```

Listing 15.2: Getriebeübersetzung als SimpleTransmission

Die Physik-Engine sieht nur das Ausgangsgelenk. Die Übersetzung bildet das Verhältnis Motorwinkel $\leftrightarrow$ Gelenkwinkel ab; die *nach* dem Getriebe verfügbaren Werte trägst du als **effort/ velocity-Limits** am Joint ein (hohes Moment, niedrige Drehzahl). Reales Restspiel und Reibung kannst du optional über die Joint-Dynamik (**damping, friction**) annähern.

Teil V

Die vier Projekte: Architektur und

rqt_graph

Kapitel 16

Projekt 1 – Stepper-Roboter mit Trajektorienbefehl vom PC

Ziel: Vom externen PC eine gewünschte Trajektorie senden, der Roboter führt sie aus. Da es ein langer Auftrag mit Fortschritt/Abbruch ist, ist die natürliche Schnittstelle eine **Action**.

16.1 ROS 2-Bausteine

Typ	Konkret
Nodes	trajectory_commander (PC, Action-Client) · controller_manager + joint_trajectory_controller + joint_state_broadcaster · Hardware-Interface (Sim: gz_ros2_control; Real: micro-ROS→ESP32)
Actions	control_msgs/FollowJointTrajectory (Goal: Bahn; Feedback: Ist-Position; Result: Erfolg)
Topics	/joint_states (Encoder/StallGuard) · /joint_trajectory_controller/... · /tf
Services	Controller laden/umschalten (/controller_manager/switch_controller)
Parameter	Gelenklimits, Getriebeübersetzung, Reglerverstärkungen

16.2 rqt_graph (vereinfacht)

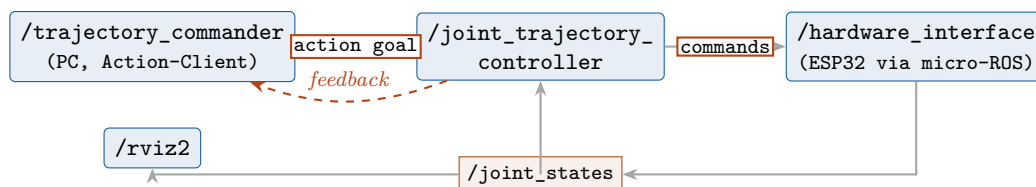


Abbildung 16.1: Projekt 1: Der PC schickt eine Trajektorie als Action-Goal; der Controller kommandiert das Hardware-Interface (ESP32) und liest /joint_states zur Regelung zurück.

Datenfluss: PC → Action-Goal (Bahn) → joint_trajectory_controller interpoliert → Sollwerte ans Hardware-Interface → ESP32 erzeugt Schritte am TMC2209, liest StallGuard/Encoder → /joint_states zurück → Regelschleife schließt sich. In Simulation ersetzt gz_ros2_control den ESP32 – der PC-Code bleibt unverändert.

Kapitel 17

Projekt 2 – Tischtennisroboter

Ziel: Ballflug per Stereokamera + Kalman schätzen, Roboter fährt an die Abschlagposition und schätzt Abschlagzeitpunkt und -trajektorie.

17.1 ROS 2-Bausteine

Typ	Konkret
Nodes	stereo_camera (Treiber) · stereo_image_proc (Disparität/Tiefe) · ball_detector (Farb-/Blob-Erkennung → 3D-Position) · ball_tracker (EKF, Flugvorhersage) · strike_planner (Abfangpunkt + Zeit + Schlagbahn) · robot_controller (Projekt-1-Stack)
Topics	/left/image_raw, /right/image_raw · /points2 · /ball/position · /ball/predicted_trajectory · /robot/target_pose
Actions	FollowJointTrajectory (Schlagbewegung)
Parameter	Kamerakalibrierung, Ballfarbe/-radius, EKF- Q/R , Tischgeometrie

17.2 rqt_graph (vereinfacht)

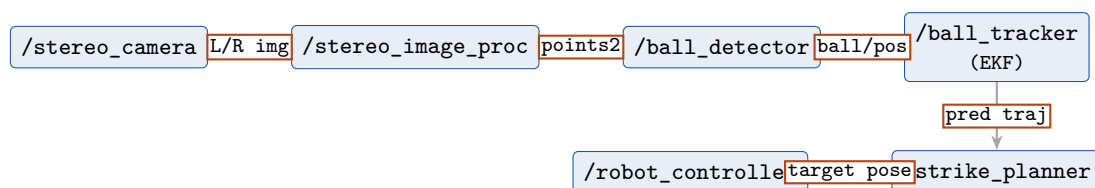


Abbildung 17.1: Projekt 2: Wahrnehmungskette von der Stereokamera über den EKF bis zum Schlagplaner und Roboter.

Kernidee: Der EKF (Kapitel 2, konstante-Beschleunigung-Modell mit Gravitation) bekommt die verrauschten 3D-Ballpositionen und extrapoliert per reiner Prädiktion bis zur Schlagebene. Daraus folgen Abfangpunkt und -zeit. **Hauptsächlich reales Problem:** Latenz und Timing – in der Simulation triviale Zeit, real kritisch.

Kapitel 18

Projekt 3 – Mobiler Roboter: Odometrie, Navigation, Bewegungserkennung

Ziel: Mit Stereokamera, Odometrie und Pfadalgorithmen navigieren und Bewegung erkennen.

18.1 ROS 2-Bausteine

Typ	Konkret
Nodes	stereo_camera · visual_odometry oder Radodometrie · robot_localization (EKF: Odom+IMU) · slam_toolbox (Karte) · nav2 (Planer+Controller) · motion_detector (optischer Fluss/Hintergrundsubtraktion)
Topics	/odom · /tf · /scan bzw. /points2 · /map · /cmd_vel
Actions	nav2_msgs/NavigateToPose
Parameter	Kostenkarte, Controller-Frequenz, EKF-Konfiguration, Roboterfußabdruck

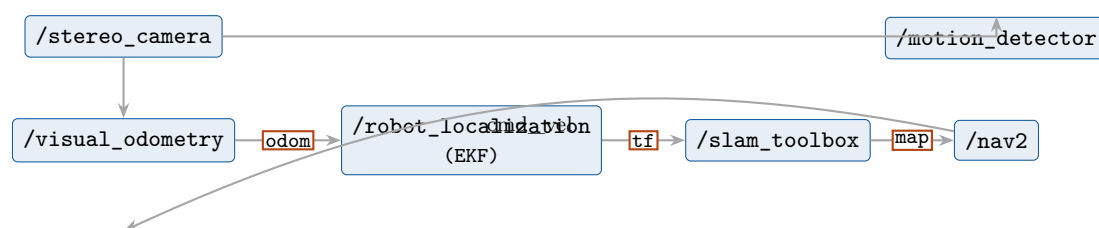


Abbildung 18.1: Projekt 3: Odometrie → EKF-Fusion → SLAM → Nav2; Bewegungserkennung als paralleler Zweig.

Kapitel 19

Projekt 4 – CAD-basierte Posenerkennung und Greifen

Ziel: Bauteile per Stereokamera über sich matchende CAD-Modelle erkennen, deren 6D-Lage bestimmen und das Greifen daran anpassen.

19.1 ROS 2-Bausteine

Typ	Konkret
Nodes	stereo_camera · part_detector (ORB/SIFT-Matching gegen gerenderte CAD-Ansichten <i>oder</i> Punktwolke+ICP) · pose_estimator (solvePnP/ICP → 6D-Pose) · moveit2 (Greifplanung) · robot_controller
Topics/Msgs	vision_msgs/Detection3DArray · Pose über /tf
Actions	MoveIt 2 MoveGroup (Planen + Ausführen)
Parameter	CAD-Modellpfade, Matching-Schwellen, ICP-Iterationen, Greifvorgaben

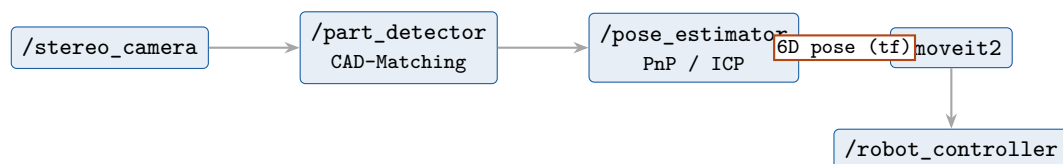


Abbildung 19.1: Projekt 4: CAD-Matching → 6D-Posenschätzung → MoveIt 2-Greifplanung, angepasst an die erkannte Werkstücklage.

19.2 Zwei Wege zur 6D-Pose

Für das CAD-Matching gibt es einen klassischen und einen lernbasierten Pfad – beide enden im selben `pose_estimator` und liefern $(\mathbf{R}, \mathbf{t})$ über /tf an MoveIt 2:

1. **Klassisch (kein Training):** ORB/SIFT-Matching gegen gerenderte CAD-Ansichten → `solvePnP`/`Ransac`, *oder* Stereo-Punktwolke + ICP gegen die CAD-Punktwolke. Schnell aufzusetzen, aber fragil bei texturlosen/glänzenden/symmetrischen Teilen.
2. **Gelernte Surface Embeddings (SurfEmb-Linie):** das im Theoriekapitel beschriebene Zwei-Turm-Netz. Robuster bei genau diesen Industrieteilen, dafür Trainingsaufwand. **Empfehlung:** klassisch starten, lernbasiert nachrüsten, wenn die Teile die klassischen Deskriptoren überfordern.

Die theoretischen Grundlagen beider Wege (Rotationsrepräsentationen, kontrastives Lernen, Two-Tower-Modell, PnP+RANSAC, grobe/feine Registrierung, PointNet++) stehen im Theoriekapitel zur 6-DoF-Posenschätzung.

19.3 Inferenz-Pipeline (lernbasiert)

1. **Detektion/Segmentierung** des Objekts im Stereobild → Bildausschnitt (RGB oder RGB-D) plus bekanntes CAD-Modell als Eingang.
2. **Embedding**: Query-Tower bettet die Pixel ein, Key-Tower die CAD-Oberflächenpunkte – in denselben Merkmalsraum.
3. **Matching** im Embedding-Raum (nächster Nachbar) → dichte 2D-3D-Korrespondenzen.
4. **Pose-Hypothesen**: PnP+RANSAC sampeln und bewerten (Scoring), beste Hypothese wählen.
5. **Verfeinerung** per ICP → finale $(\mathbf{R}, \mathbf{t})$, Modell→Roboterframe.

Punktwolken-Zusatzschritte (3D-3D-Pfad): Hintergrund entfernen, Voxel-Downsampling, statistischer Ausreißerfilter – vor der Registrierung.

Labels sind quasi gratis

Das CAD-Modell wird in bekannten Posen gerendert → Ground-Truth-Pose *und* die Pixel-Punkt-Korrespondenzen liegen automatisch vor. Kein manuelles Annotieren, keine Symmetrie-Labels. Das synthetische Training wird per **Domain Randomization** und PBR-Rendering (Isaac Sim) gegen den Sim-to-Real-Gap abgehärtet, optional mit kleinem realem Feintuning-Satz.

19.4 Stereo als metrische Skala

Die Stereokamera ist hier nicht nur Bildquelle: Sie löst die **Skalenmehrdeutigkeit** der Mono-Ansicht. Tiefe geht entweder als RGB-D-Zusatzkanal ins Netz oder wird zur Punktwolke rückprojiziert. Erst dadurch steht $(\mathbf{R}, \mathbf{t})$ in echten Millimetern – die Voraussetzung dafür, dass MoveIt 2 überhaupt greifen kann. **Vorsicht**: glänzendes Metall liefert ungültige Tiefe; saubere Kalibrierung, Rektifizierung und Tiefen-Entrauschung sind Pflicht.

19.5 Bewegte Teile: Pose-Tracking mit Kalman

Liegt das Teil nicht still (Förderband, armmontierte Kamera), ist die bildweise Netz-Pose veräuscht und zitterig. Ein **Kalman-Filter** (vgl. Projekt 1/3 und das Kalman-Kapitel) macht aus Einzelschuss ein *Tracking*: Zustand = Pose (ggf. + Geschwindigkeit), Messung = bildweise Netz-Pose; Prädiktion → Korrektur glättet Jitter, verwirft Ausreißer-Frames und überbrückt kurze Verdeckungen. Da Rotationen nichtlinear sind: EKF/UKF oder Filtern auf einer geeigneten Rotationsdarstellung. *Dieselbe Kalman-Kompetenz wie beim Tischtennisball – nur auf die 6D-Pose statt die Ballposition angewandt.*

19.6 Hyperparameter und Evaluation

Stellgröße	Wirkung
Embedding-Dimension	Ausdrucksstärke des Merkmalsraums
Kontrastive Temperatur τ	Schärfe der Zuordnung im Verlust
#Negative / Backbone-Tiefe	Trennschärfe bzw. Kapazität
LR / Batch / Optimierer	Trainingsdynamik

Augmentierungsstärke		Robustheit gegen Sim-to-Real-Gap
RANSAC-Iterationen	+	Ausreißerrobustheit der Posenlösung
Inlier-Schwelle		
#Pose-Hypothesen	/	Genauigkeit vs. Laufzeit
Crop-Auflösung		

Metriken: ADD / ADD-S (ADD-S für symmetrische Teile), geodätischer Rotations- und euklidischer Translationsfehler, RMSE; im Forschungskontext die BOP-Metriken (VSD, MSSD, MSPD → Average Recall) auf Datensätzen wie T-LESS/ITODD. **Industrielle KPIs:** Taktzeit/Latenz, Pick-Erfolgsrate, Falsch-Positiv-Rate – das sind die Zahlen, die im Maschinenbeladungs-Einsatz zählen.

Teil VI

Stereokamera in ROS 2 – Praxis mit Code

Kapitel 20

Kamera anbinden, Frame splitten

Die synchronisierte ELP-Kamera liefert ein side-by-side-Bild. Erst den Treiber starten, dann links/rechts trennen.

```

1  ros2 run v4l2_camera v4l2_camera_node \
2    --ros-args -p video_device:=/dev/video0 \
3    -p image_size:="[2560,720]"
4  ros2 topic list                                # /image_raw sollte erscheinen
5  ros2 run rqt_image_view rqt_image_view         # Bild ansehen

```

Listing 20.1: Treiber starten und Topics prüfen

```

1  import rclpy
2  from rclpy.node import Node
3  from sensor_msgs.msg import Image
4  from cv_bridge import CvBridge
5
6  class StereoSplitter(Node):
7      def __init__(self):
8          super().__init__('stereo_splitter')
9          self.bridge = CvBridge()
10         self.pub_l = self.create_publisher(Image, '/left/image_raw', 10)
11         self.pub_r = self.create_publisher(Image, '/right/image_raw', 10)
12         self.create_subscription(Image, '/image_raw', self.cb, 10)
13
14         def cb(self, msg):
15             frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
16             w = frame.shape[1] // 2
17             left, right = frame[:, :w], frame[:, w:]
18             hdr = msg.header # keep timestamp so stereo stays synchronized
19             lm = self.bridge.cv2_to_imgmsg(left, 'bgr8'); lm.header = hdr
20             rm = self.bridge.cv2_to_imgmsg(right, 'bgr8'); rm.header = hdr
21             self.pub_l.publish(lm)
22             self.pub_r.publish(rm)
23
24         def main():
25             rclpy.init()
26             rclpy.spin(StereoSplitter())
27             rclpy.shutdown()

```

Listing 20.2: Knoten: side-by-side Frame in /left und /right splitten

Kapitel 21

Kalibrieren und Tiefe erzeugen

```
1  # Kalibrieren: erzeugt left.yaml / right.yaml
2  #           mit Intrinsik + Extrinsik
3  ros2 run camera_calibration cameracalibrator \
4  --size 8x6 --square 0.025 \
5  --ros-args -r left:=/left/image_raw \
6             -r right:=/right/image_raw
7
8  # Rektifizierung + Disparitaet -> Tiefe / Punktwolke
9  ros2 launch stereo_image_proc stereo_image_proc.launch.py
10 # liefert /disparity und /points2
```

Listing 21.1: Stereokalibrierung (Schachbrett) und Tiefenpipeline

Kapitel 22

Kalman-Filter-Knoten (Ballverfolgung)

```

1  import numpy as np, rclpy
2  from rclpy.node import Node
3  from geometry_msgs.msg import PointStamped
4
5  class BallTracker(Node):
6      def __init__(self):
7          super().__init__('ball_tracker')
8          dt = 1/60.0
9          self.x = np.zeros((6,1))          # [x y z vx vy vz]
10         self.P = np.eye(6) * 1.0
11         self.F = np.eye(6); self.F[:3,3:] = np.eye(3)*dt
12         self.g = np.array([[0],[0],[-9.81]])
13         self.Bu = np.vstack([0.5*dt**2*self.g, dt*self.g])
14         self.H = np.zeros((3,6)); self.H[:,3:] = np.eye(3)
15         self.Q = np.eye(6)*0.01
16         self.R = np.eye(3)*0.0025          # ~5 cm Messrauschen
17         self.sub = self.create_subscription(
18             PointStamped, '/ball/position', self.update, 10)
19         self.timer = self.create_timer(dt, self.predict)
20
21     def predict(self):
22         self.x = self.F @ self.x + self.Bu
23         self.P = self.F @ self.P @ self.F.T + self.Q
24
25     def update(self, msg):
26         z = np.array([msg.point.x, msg.point.y, msg.point.z])
27         S = self.H @ self.P @ self.H.T + self.R
28         K = self.P @ self.H.T @ np.linalg.inv(S)    # Kalman-Gain
29         self.x = self.x + K @ (z - self.H @ self.x)
30         self.P = (np.eye(6) - K @ self.H) @ self.P
31
32     def predict_intercept(self, z_plane):
33         # reine Praediktion bis zur Schlagebene (kein Messupdate)
34         x = self.x.copy()
35         for _ in range(120):
36             x = self.F @ x + self.Bu
37             if x[2,0] <= z_plane:
38                 return x[:3].flatten()
39         return None

```

Listing 22.1: EKF/KF-Knoten: 3D-Position rein, geglättete Schätzung + Vorhersage raus

Kapitel 23

Merkmalerkennung ohne KI-Training

```

1 import cv2
2
3 orb = cv2.ORB_create(nfeatures=1500)
4 # Referenzansicht des Bauteils (z.B. aus CAD-Render) einmalig beschreiben:
5 kp_ref, des_ref = orb.detectAndCompute(ref_gray, None)
6 matcher = cv2.BFMatcher(cv2.NORM_HAMMING)
7
8 def detect(frame_gray):
9     kp, des = orb.detectAndCompute(frame_gray, None)
10    if des is None:
11        return []
12    raw = matcher.knnMatch(des_ref, des, k=2)
13    good = [m for m, n in raw if m.distance < 0.75 * n.distance] # Lowe
14    return kp, good
15
16 # 6D-Pose ueber 2D-3D-Korrespondenzen (CAD-Punkte <-> Bildpunkte):
17 # ok, rvec, tvec = cv2.solvePnPRansac(obj_pts_3d, img_pts_2d,
18 #                                     K, distCoeffs)

```

Listing 23.1: ORB-Erkennung und Matching (klassisch, kein Training nötig)

Für **Bewegungserkennung** (Projekt 3) genügt Hintergrundsubtraktion:

```

1 bg = cv2.createBackgroundSubtractorMOG2(detectedShadows=False)
2 mask = bg.apply(frame) # bewegte Pixel
3 contours, _ = cv2.findContours(
4     mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
5 movers = [c for c in contours if cv2.contourArea(c) > 500]

```

Listing 23.2: Bewegungserkennung ohne Merkmale

Kapitel 24

Echtzeit-Stereoverarbeitung: Scheduling, Frame-Dropping und Zustandsautomat

Der Balltracker ist **zeitkritisch**: Bei 60 fps hast du pro Frame nur $\approx 16,7$ ms, um zu splitten, zu rektifizieren, den Ball in beiden Bildern zu finden, zu triangulieren und den Kalman-Filter zu aktualisieren. Dauert ein Frame länger, läuft die Verarbeitung der Realität hinterher. Dieses Kapitel zeigt, wie man damit umgeht: Frames gezielt verwerfen, die Prädiktion von der Messung entkoppeln und die Ablauflogik als Zustandsautomat strukturieren.

24.1 Das Zeitbudget eines Frames

Die Pipeline besteht aus Stufen mit je eigenem Latenzanteil:

Stufe	typ. Latenz	Anmerkung
Belichtung + USB-Transfer	2–8 ms	von der Kamera vorgegeben
Split + Rektifizierung	1–3 ms	einmalige Remap-LUT
Balldetektion (sparse, beide Bilder)	2–10 ms	der teure, schwankende Teil
Triangulation + KF-Update	< 1 ms	billig

Es gibt **zwei verschiedene Fehlermodi**, die man nicht verwechseln darf:

Latenz (zu alt).

Ein einzelner Frame ist verarbeitet, aber das Ergebnis kommt zu spät – die Schätzung beschreibt einen vergangenen Ballort.

Durchsatz (Stau).

Frames kommen schneller, als du sie verarbeitest. Ohne Gegenmaßnahme wächst die Warteschlange unbegrenzt, und die Latenz steigt mit jeder Sekunde weiter an (*Bufferbloat*).

Die Kernregel für den Kalman-Filter

Ein **veralteter** Messwert ist schlechter als *gar kein* Messwert. Der Filter kann über reine Prädiktion problemlos einige Frames überbrücken (vgl. Kalman-Knoten). Also: lieber den alten Frame **verwerfen** und auf den nächsten aktuellen warten, als einen veralteten Messwert ins Update zu geben. **Niemals Frames aufstauen.**

24.2 Frame-Dropping: DDS macht die Arbeit

Der wichtigste Hebel ist die **QoS**: Mit `KEEP_LAST`, Tiefe 1 und `BEST_EFFORT` verwirft DDS selbst alte Bilder – während dein Callback rechnet, überschreibt jeder neue Frame den wartenden, und du bekommst nach dem Callback stets den *neuesten* statt des nächst-ältesten. Genau das ist das `sensor_data`-QoS-Profil.

```
1 from rclpy.qos import qos_profile_sensor_data
2 # entspricht: BEST_EFFORT + KEEP_LAST + depth=1
3 self.create_subscription(
4     Image, '/left/image_raw', self.on_frame, qos_profile_sensor_data,
5     callback_group=self.cb_detect)
```

Listing 24.1: QoS für Bild-Topics: nur der neueste Frame zählt

Warum nicht RELIABLE? `RELIABLE` lässt DDS verlorene Frames neu senden – bei einem Hochfrequenz-Bildstrom erzeugt das genau den Rückstau, den du vermeiden willst. `RELIABLE` gehört auf das *Ergebnis* (die 3D-Ballposition), `BEST_EFFORT` auf den Bildstrom.

Als zweite Verteidigungslinie verwirfst du im Callback explizit zu alte Frames anhand des Zeitstempels – das fängt auch den Fall ab, dass DDS dir doch einen leicht veralteten Frame zustellt:

```
1 def on_frame(self, msg):
2     now = self.get_clock().now()
3     stamp = rclpy.time.Time.from_msg(msg.header.stamp)
4     age_s = (now - stamp).nanoseconds * 1e-9
5     if age_s > self.max_age_s:           # z. B. 0.020 s = ein Frame bei 60 fps
6         self.dropped += 1
7         return                          # verwerfen statt verspaetet
8                                         # verarbeiten
9     self.process(msg)                   # detektieren -> triangulieren -> KF-
                                         # Update
```

Listing 24.2: Stale-Gating: Frame anhand seines Alters verwerfen

24.3 Scheduling: Executor und Callback-Gruppen

Standardmäßig serialisiert der `SingleThreadedExecutor` alle Callbacks: ein langsamer Detektions-Callback **blockiert dann auch den Prädiktions-Timer**. Für zeitkritischen Code trennst du die Anliegen über einen `MultiThreadedExecutor` und **Callback-Gruppen**:

- `MutuallyExclusiveCallbackGroup`: Callbacks dieser Gruppe laufen nie gleichzeitig – richtig für den CPU-gebundenen Detektor (eine Instanz zur Zeit, zusammen mit Tiefe 1 ergibt das automatische Frame-Dropping).
- Ein *eigener* Timer-Callback in einer *eigenen* Gruppe für die Prädiktion – so läuft sie in festem Takt weiter, selbst während die Detektion noch an einem Frame rechnet.

```
1 from rclpy.executors import MultiThreadedExecutor
2 from rclpy.callback_groups import MutuallyExclusiveCallbackGroup
3
4 class Perception(Node):
5     def __init__(self):
6         super().__init__('perception')
7         self.cb_detect = MutuallyExclusiveCallbackGroup() # teuer,
8                                                             schwankend
9         self.cb_predict = MutuallyExclusiveCallbackGroup() # billig,
10                                                             deterministisch
11         self.create_subscription(Image, '/left/image_raw', self.on_frame,
```

```

10         qos_profile_sensor_data, callback_group=self.
11             cb_detect)
12         self.create_timer(1/120.0, self.predict, callback_group=self.cb_predict
13     )
14
15 def main():
16     rclpy.init()
17     node = Perception()
18     ex = MultiThreadedExecutor(num_threads=3)    # detect / predict / reserve
19     ex.add_node(node); ex.spin()

```

Listing 24.3: Prädiktion und Detektion in getrennten Gruppen, parallel ausgeführt

Prädiktion schneller als die Kamera. Der Predict-Timer darf ruhig mit 120 Hz laufen, obwohl die Kamera nur 60 fps liefert – Prädiktion ist billig und liefert zwischen den Messungen eine glatte, aktuelle Schätzung. Die Messung (Update) kommt ereignisgesteuert, wann immer ein *frischer* Frame durchkommt. Das ist genau die Predict/Update-Aufteilung aus dem Kalman-Knoten, nur sauber auf zwei Threads gelegt.

Veraltete und vertauschte Messungen (Out-of-Order)

Kommt eine Messung mit einem Zeitstempel *vor* dem aktuellen Filterstand an (z.B. weil ein langsamer Detektions-Thread sie spät abliefert), sind zwei Wege sauber: (a) sie schlicht **verwerfen** (einfach, meist ausreichend), oder (b) ein **Out-of-Sequence-Update** – Filter auf den Mess-Zeitpunkt zurückrollen, einarbeiten, wieder vorrollen. Für den Tischtennisball reicht (a). Hilfreich, wenn linkes/rechtes Bild synchron sein müssen: `message_filters.ApproximateTimeSynchronizer`.

24.4 Ablauflogik als Zustandsautomat

Die Frage “was tue ich pro Frame” hängt vom **Zustand** ab: Solange kein Ball da ist, suchst du; sobald genug Punkte einer Flugbahn vorliegen, rechnest du den Auftreffpunkt; ist der Schlag ausgelöst, fährst du ihn ab. Diese Logik gehört nicht verstreut in den Bild-Callback, sondern in einen expliziten **Zustandsautomaten**, der die Wahrnehmungs-Rate von der Entscheidungs-Rate entkoppelt.

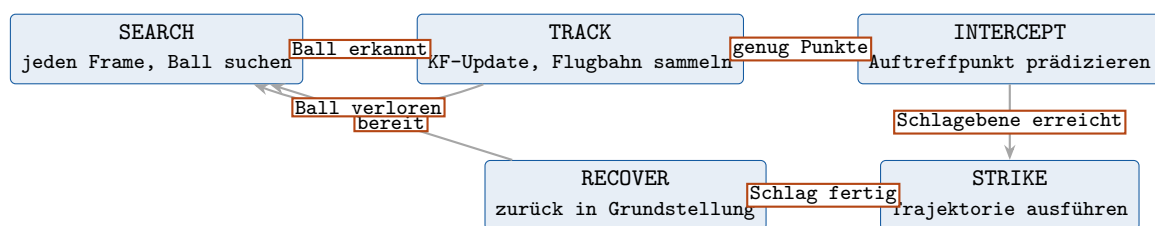


Abbildung 24.1: Zustandsautomat des Balltrackers. Die Stereo-Detektion treibt die Übergänge; die Prädiktion läuft zustandsübergreifend im eigenen Timer weiter.

Umsetzung in ROS 2 – drei Stufen nach Komplexität:

Eigenes Enum im Knoten.

Für eine überschaubare Logik wie diese das Pragmatischste: ein Zustandsfeld plus `match/if`. Kein Framework, voll unter Kontrolle.

YASMIN.

ROS 2-native Zustandsmaschinen-Bibliothek (Python/C++) mit Introspektion – sinnvoll, wenn die Zustände wachsen.

Behavior Trees.

`BehaviorTree.CPP` (wie in Nav2) oder `py_trees_ros` – für komplexe, wiederverwendbare Verhal-

tensbäume mit Fallbacks; für den reinen Balltracker meist überdimensioniert.

```

1 from enum import Enum, auto
2
3 class S(Enum):
4     SEARCH = auto(); TRACK = auto(); INTERCEPT = auto(); STRIKE = auto();
5     RECOVER = auto()
6
7 class Brain(Node):
8     def __init__(self, kf):
9         super().__init__('brain')
10        self.kf = kf; self.state = S.SEARCH; self.hits = 0
11        # eigener Takt: Entscheidung entkoppelt von der Frame-Rate
12        self.create_timer(1/200.0, self.tick)
13
14    def on_measurement(self, ok):
15        # vom Detektor pro frischem Frame
16        self.hits = self.hits + 1 if ok else 0
17
18    def tick(self):
19        if self.state is S.SEARCH and self.hits > 0:
20            self.state = S.TRACK
21        elif self.state is S.TRACK:
22            if self.hits == 0:
23                Ball verloren
24                self.state = S.SEARCH
25            elif self.hits >= self.min_track:
26                self.state = S.INTERCEPT
27        elif self.state is S.INTERCEPT:
28            p = self.kf.predict_intercept(self.z_plane) # reine Praediktion
29            if p is not None:
30                self.send_goal(p); self.state = S.STRIKE
31        elif self.state is S.STRIKE and self.motion_done():
32            self.state = S.RECOVER
33        elif self.state is S.RECOVER and self.at_home():
34            self.hits = 0; self.state = S.SEARCH

```

Listing 24.4: Zustandsautomat als Enum – Entscheidung getrennt vom Bild-Callback

Das Architektur-Prinzip in einem Satz

Drei entkoppelte Takte statt einer verschachtelten Schleife: **Detektion** (ereignisgesteuert, verwirft alte Frames per QoS), **Prädiktion** (schneller Timer, läuft immer), **Entscheidung** (eigener Timer, fährt den Zustandsautomaten) – über einen `MultiThreadedExecutor` mit getrennten Callback-Gruppen, damit kein langsamer Frame die anderen beiden blockiert.

Teil VII

Tischtennisroboter: Mechanik, Kamera-Architektur und CAD/URDF

Kapitel 25

Kamera-Architektur – wo wird gerechnet?

Lohnt es sich, die Stereokamera an den ESP32 / Raspberry Pi Zero zu hängen und die Rohdaten per ROS-Service oder einfachem UDP/WLAN weiterzuleiten – oder entstehen Zeit-/Rechenleistungsprobleme?

25.1 Die Bandbreitenrechnung entscheidet

Die synchronisierte Stereokamera liefert ein side-by-side-Bild (z. B. $2 \times 1280 \times 720$). Was das pro Sekunde bedeutet:

Datenstrom	pro Frame	bei 60 fps
Stereo roh, YUYV (2 B/px), 2560×720	3,69 MB	≈ 1769 Mbit/s
Stereo MJPEG-komprimiert ($\approx 1:10$)	0,37 MB	≈ 177 Mbit/s
Nur 3D-Ballposition (Header + 3 Floats)	≈ 80 B	$\approx 0,1$ Mbit/s

Dem gegenüber die realistischen Linkkapazitäten:

Verbindung	real nutzbar
ESP32 WLAN (TCP, real)	$\approx 10\text{--}30$ Mbit/s
Pi Zero (2,4 GHz WLAN)	einige zehn Mbit/s, hoher Jitter
Pi 4 WLAN (Dualband)	$\approx 100\text{--}300$ Mbit/s, mit Jitter
Pi 4 Gigabit-Ethernet (Kabel)	1000 Mbit/s, deterministisch
USB 2 / USB 3	480 / 5000 Mbit/s

25.2 Verdikt pro Plattform

ESP32 – nein.

Zwei harte Gründe: (1) Der ESP32 ist *kein* USB-Host für UVC-Kameras. (2) 177–1769 Mbit/s liegen um Größenordnungen über seinem WLAN-Durchsatz. Der ESP32 gehört auf die **Echtzeit-Motorebene**, nicht an die Kamera.

Pi Zero – technisch möglich, aber schlechte Idee.

Der Zero 2 W kann mit OTG-Adapter eine UVC-Kamera einlesen, aber: schwache CPU, nur 2,4-GHz-WLAN, und er wäre allein mit dem Datenschaukeln gesättigt. Für einen *latenzkritischen* Balltracker ist WLAN-Jitter Gift.

Pi 4 – ja, und das ist der richtige Ansatz.

Der Pi 4 hat USB 3, Gigabit-Ethernet und eine brauchbare Quad-Core-CPU. Er kann die Kamera einlesen *und die Bildverarbeitung lokal machen*.

Das richtige Muster: am Rand rechnen, nur das Ergebnis senden

Schiebe nicht die Pixel durchs Netz, sondern die **Erkenntnis**. Lass den Pi 4 (direkt an der Kamera) die Stereo-Erkennung machen und nur die **3D-Ballposition** publizieren – das sind $\approx 0,1$ Mbit/s, trivial über jedes Netz. ROS 2/DDS transportiert das Topic netzwerktransparent zum PC; du brauchst kein manuelles UDP. Für deterministische Latenz: **Pi 4 per Kabel-Gigabit-Ethernet** an den PC.

25.3 Sparse statt dense: der Ball braucht keine volle Tiefenkarte

Eine dichte Disparitätskarte (SGBM über das ganze Bild) ist teuer. Für die Ballverfolgung genügt **sparse Triangulation**: den Ball (heller Blob) in *beiden* rektifizierten Bildern detektieren, je Bild seine Pixelspalte bestimmen, daraus die Disparität *nur dieses einen Punktes* und über $Z = fb/d$ die Tiefe. Das läuft auf dem Pi 4 locker bei 60–120 fps.

25.4 ROS-Service, UDP oder Topic?

- **ROS-Service: falsch** für Streaming. Services sind synchrones Request/Response (blockierend, one-shot) – für kontinuierliche Bild-/Positionsströme ungeeignet.
- **ROS-Topic: richtig**. Ergebnis (Ballposition) als RELIABLE-Topic; falls du doch Bilder überträgst, image_transport mit compressed und QoS BEST_EFFORT.
- **Rohes UDP/GStreamer: nur als letzter Ausweg**, wenn du ROS für reines Bildstreaming umgehen willst – du verlierst dann die ROS-Integration. Für berechnete Ergebnisse unnötig.

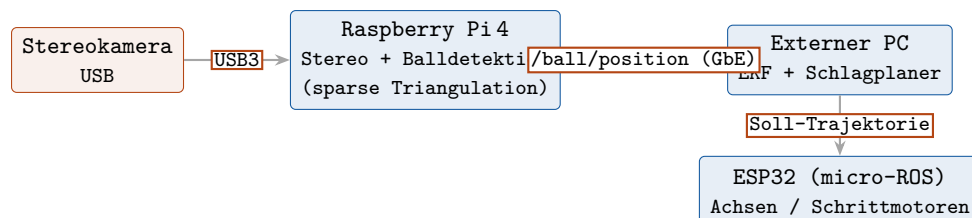


Abbildung 25.1: Empfohlene Architektur: schwere Pixelarbeit am Pi 4 nahe der Kamera, nur die destillierte Ballposition wandert (kabelgebunden) zum PC.

Kapitel 26

Mechanischer Aufbau: Portalsystem (Gantry)

26.1 Grundkonzept

Statt eines Delta-Roboters ein **kartesisches Portalsystem**: große Linearachsen positionieren den Schläger grob im Raum, ein kleiner Arm am Ende führt die Schlagbewegung aus und stellt den Schlägerwinkel. Bezug Tischtennisplatte (ITTF): 2,74 m × 1,525 m, Höhe 0,76 m, Netz 15,25 cm. Die **X-Achse** überspannt die Plattenbreite (1,525 m).

26.2 Geschachtelte Achsen

- **X-Achse** (Plattenbreite, horizontal): zwei parallele Führungsstäbe, Zahnriemenantrieb, X-Schlitten. Bauprinzip wie die Bambu-Lab-P2S-X-Achse – *glatte Hohlstahl-Stäbe + GT2-Zahnriemen*, leichter Schlitten.
- **Z-Achse** (Höhe, vertikal): nach demselben Prinzip, montiert *im* X-Schlitten. Der Abstand der X-Führungsstäbe definiert den Bauraum und damit den **Hub der Z-Achse**.
- **Y-Achse** (Tiefe): entweder starr (fester Ausleger) oder als dritte Linearachse auf dem Z-Schlitten.
- **Schlagarm**: auf dem Y/Z-Schlitten ein kleiner Roboterarm mit Schlägereinspannung.

Zahnriemen, nicht Keilriemen

Für *Positionierung* brauchst du einen **Zahnriemen** (GT2-Zahnriemen), keinen Keilriemen (V-Riemen). Ein Keilriemen überträgt per Reibschluss und *schlupft* – damit verlierst du Positionsgenauigkeit. Zahnriemen sind formschlüssig, schlupffrei und der Standard in 3D-Druckern/Plottern.

26.3 Antriebstopologie und Hardware-Anschluss

Element	Umsetzung
X-/Z-/Y-Motoren	NEMA-17, möglichst <i>rahmenfest</i> (Riemen treibt Schlitten)
Treiber	TMC2209 (UART) je Achse, StallGuard als Last-/Endlagenhinweis
Endschalter	optische/mechanische Endstops je Achse zum Homing
Strom/Spannung	INA226-Modul je Treiberzweig (I ² C), optional
Controller	ESP32 (micro-ROS): bündelt STEP/DIR bzw. UART, liest Endstops + INA226

Versorgung	24 V Netzteil für Motoren, getrennte 5 V-Logik; Sternmasse
Schlagarm-Motoren	klein, idealerweise basisnah (Riemen-/Seilzug zu distalen Gelenken)

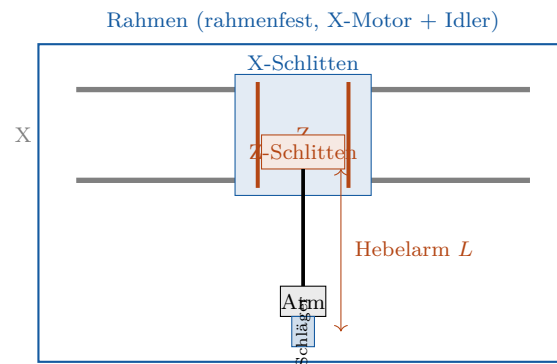


Abbildung 26.1: Schematischer Frontblick: X-Schlitten trägt die im Rod-Abstand geschachtelte Z-Achse; der Schlagarm hängt als Ausleger (Hebelarm L) darunter.

Kapitel 27

Das Hebelarm- und Schwingungsproblem

Der Arm hängt mit Abstand L unter der X-Achse. Bei harten Beschleunigungen zur Zielposition entstehen Reaktionskräfte und -momente, die den Ausleger zum Schwingen anregen.

27.1 Die Physik

Beschleunigt der Schlitten mit a , wirkt auf die bewegte Masse m die Trägheitskraft $F = m a$. Über den Hebelarm L ergibt das am Anbindungspunkt ein **Biegemoment**

$$M = F \cdot L = m a L. \quad (27.1)$$

Der Ausleger verhält sich näherungsweise wie ein einseitig eingespannter Balken (Kragträger) mit Steifigkeit $k = \frac{3EI}{L^3}$ und einer ersten **Eigenfrequenz**

$$f_1 = \frac{1}{2\pi} \sqrt{\frac{k}{m_{\text{eff}}}} = \frac{1}{2\pi} \sqrt{\frac{3EI}{m_{\text{eff}} L^3}}. \quad (27.2)$$

E = E-Modul, I = Flächenträgheitsmoment des Auslegerquerschnitts. Wichtig: f_1 fällt mit $L^{3/2}$ – ein längerer Ausleger ist drastisch weicher. Anregungsanteile der Bewegung nahe f_1 führen zu Resonanz und langem Nachschwingen, was die Schlaggenauigkeit ruiniert.

27.2 Gegenmaßnahmen

1. **Bewegte Masse minimieren:** Motoren rahmenfest (Riemen-/Seilzug zu den Armgelenken, CoreXY-Logik), Arm aus Aluminium/CFK, Schwerpunkt nah an der Achse.
2. **Steifigkeit maximieren:** große Stab-/Schienenquerschnitte (gestützte Linearschienen statt dünner Stäbe), Doppelführung, kurzer Ausleger, triangulierter Rahmen. f_1 deutlich *über* die Betriebsfrequenzen legen.
3. **Ruckbegrenzte Bewegungsprofile (S-Kurve):** sanftes Beschleunigen regt hohe Frequenzen weniger an.
4. **Input Shaping / Resonanzkompensation:** der Steuerbefehl wird so vorgeformt, dass die Anregung bei f_1 ausgelöscht wird – genau das, was Bambu/Klipper mit einem Beschleunigungssensor (ADXL345) als “resonance compensation” tun. Das ist die wirksamste softwareseitige Maßnahme.
5. **Dämpfung:** Strukturdämpfung, ggf. ein abgestimmter Tilger (tuned mass damper); Riemenvorspannung erhöhen (Riemendehnung ist Teil der Nachgiebigkeit).

Anknüpfung an den FEM-Hintergrund

Bestimme f_1 und die Modenformen per **Modalanalyse** (SolidWorks Simulation / FEM). Das liefert dir genau die Frequenz, auf die du das Input Shaping abstimmst, und zeigt, welche Strukturteile du versteifen musst. Das verbindet FEM-Erfahrung direkt mit der Reglerauslegung.

Kapitel 28

Simulierbarkeit in Gazebo und RViz

Lassen sich Anfahren der Zielposition, Schlagbewegung und die induzierten Kräfte mit Gazebo/RViz abbilden?

28.1 Was Gazebo kann – und was nicht

Gazebo bildet ab (Starrkörperdynamik)	Gazebo bildet <i>nicht</i> ab
Anfahren der Zielposition (prismatische + rotatorische Gelenke)	Strukturelle Biegung/Durchbiegung der Stäbe
Schlagbewegung des Arms	Riemendehnung als kontinuierliche Elastizität
Gelenkmomente/-kräfte (auslesbar)	Modale Resonanz/Eigenschwingung der Struktur
Dynamische Kopplung (Armträgheit reagiert auf Portal)	Rahmenflattern, Mikrovibrationen
Ball-Schläger-Kontakt + Reaktionskräfte	exakte Spin-/Absprungphysik (nur genähert)
Motormoment-Budgetierung	

Der Kernpunkt

Gazebo behandelt Links als **starre Körper**. Die Schwingung, die dich sorgt, kommt aber großteils aus *struktureller Nachgiebigkeit* (Stabbiegung, Riemendehnung, Rahmenflex) – starre Links biegen sich nicht, also zeigt Standard-Gazebo diese Resonanz *nicht*.

28.2 Wege, die Nachgiebigkeit doch abzubilden

- **Klumpen-Nachgiebigkeit in Gazebo:** ein steifes Feder-Dämpfer-Gelenk zwischen Segmenten einfügen (Riemendehnung als prismatische Feder, oder ein nachgiebiges Drehgelenk am Auslegerfuß). Das ist eine *Näherung* der ersten Mode, kein vollständiges Modalverhalten.
- **Echte Strukturvibration:** FEM / flexible Mehrkörpersimulation – SolidWorks Simulation (modal), MATLAB **Simscape Multibody** mit flexiblen Körpern (reduzierte FEM-Modelle), Adams. Co-Simulation mit ROS möglich.
- **RViz:** reine Visualisierung – zeigt Bewegung und TF, rechnet *keine* Physik. Kräfte kannst du als Marker visualisieren, wenn ein anderer Knoten sie berechnet/publiziert.

Empfohlene Aufteilung

Gazebo für Regelung, Trajektorientracking, Momentenbudget und Ball-Schläger-Kontakt. **FEM/Simscape** für die strukturelle Resonanz (f_1 , Modenformen). Beides verbindet sich über die Input-Shaping-Frequenz: FEM liefert f_1 , der ROS-Regler nutzt es zur Schwingungsunterdrückung.

Kapitel 29

CAD-Module und URDF

29.1 Notwendige CAD-Module mit Designanforderungen

Modul	Funktion	Designanforderung
Rahmen / Basis	trägt X-Achse + X-Motor, ortsfest	sehr steif (Alu-Profil 3030/4040), trianguliert, verankert; URDF-Wurzel
X-Achse	horizontale Verfahrachse über Plattenbreite	Hohlstahl-Stäbe + GT2-Riemen; Motor rahmenfest; Stablänge > 1,6 m
X-Schlitten	trägt + verfährt die Z-Achse	leicht aber biegesteif; Stababstand = Z-Hub
Z-Achse	vertikale Verfahrachse	gleiches Prinzip; Hub = X-Stababstand
Z-Schlitten	trägt Y/Arm	geringe Masse (bewegte Masse!)
Y-Achse (optional)	Tiefenzustellung	nur falls nötig; sonst starrer Ausleger
Armbasis	Anbindung Arm an Schlitten	steife Einspannung; kurzer Hebel L ; hält 1. Armmotor
Armglieder (2–3 DOF)	Schwung + Schlägerwinkel	leicht (Alu/CFK); Motoren basisnah; Reichweite + Geschwindigkeit für Schlag
Schlägereinspannung	hält den Schläger	starr, definierte Schlägerebene, geringe Masse
Endeffektor-Frame	Schlagpunkt + Normale	TF-Frame: Schlägerflächenmitte + Flächennormale
Kleinteile	Motorhalter, Riemenspanner, Idler, Endstop-Halter	druckbar/CNC; auf Steifigkeit der Lastpfade achten

29.2 Kinematische Kette

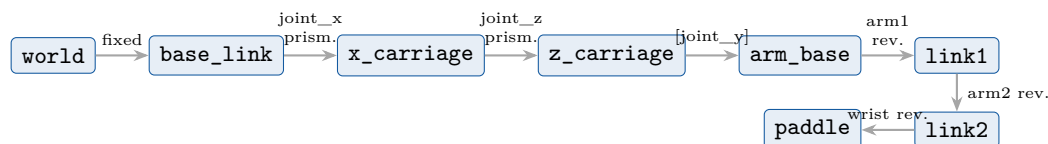


Abbildung 29.1: Kinematische Kette: drei (optional vier) Linear-/Drehübergänge bis zur Armbasis, dann der Schlagarm bis zum Schläger.

29.3 URDF/Xacro-Gerüst


```

1  <!-- Wurzel: Rahmen fest mit der Welt -->
2  <link name="base_link"> <!-- inertial/visual/collision --> </link>
3
4  <!-- X-Achse: linear ueber die Plattenbreite -->
5  <joint name="joint_x" type="prismatic">
6    <parent link="base_link"/> <child link="x_carriage"/>
7    <origin xyz="0 0 1.0" rpy="0 0 0"/>
8    <axis xyz="1 0 0"/>
9    <limit lower="0" upper="1.5" effort="60" velocity="3.0"/>
10 </joint>
11 <link name="x_carriage"> <!-- traegt die Z-Achse --> </link>
12
13 <!-- Z-Achse: linear vertikal, Hub = X-Stababstand -->
14 <joint name="joint_z" type="prismatic">
15   <parent link="x_carriage"/> <child link="z_carriage"/>
16   <axis xyz="0 0 1"/>
17   <limit lower="-0.4" upper="0.0" effort="50" velocity="2.5"/>
18 </joint>
19 <link name="z_carriage"> ... </link>
20
21 <!-- Armbasis (Y optional als prismatic dazwischen) -->
22 <joint name="joint_arm_base" type="fixed">
23   <parent link="z_carriage"/> <child link="arm_base"/>
24   <origin xyz="0 0.05 0" rpy="0 0 0"/>
25 </joint>
26 <link name="arm_base"> ... </link>
27
28 <!-- Schlagarm -->
29 <joint name="joint_arm1" type="revolute">
30   <parent link="arm_base"/> <child link="link1"/>
31   <axis xyz="0 1 0"/>
32   <limit lower="-2.0" upper="2.0" effort="8" velocity="12"/>
33 </joint>
34 <link name="link1"> ... </link>
35 <!-- joint_arm2 (revolute), joint_wrist (revolute) analog -->
36
37 <!-- Schlaeger-Endeffektor -->
38 <joint name="joint_paddle" type="fixed">
39   <parent link="link2"/> <child link="paddle"/>
40   <origin xyz="0 0 0.12" rpy="0 0 0"/>
41 </joint>
42 <link name="paddle"> ... </link>

```

Listing 29.1: Gantry + Schlagarm als URDF (gekürzt)

Riemenübersetzung der Linearachsen

Eine prismatische Riemenachse wandelt Motordrehung in Linearweg: *Weg pro Umdrehung* = Zähnezahl $\times$ Riementeilung. Ein 20-Zahn-GT2-Rad (Teilung 2 mm) ergibt $20 \times 2 = 40$ mm/Umdrehung. Mit 200×16 Microsteps folgt eine Auflösung von $40/3200 = 0,0125$ mm/Microstep. Diese Umrechnung Motor $\leftrightarrow$ mm gehört ins Hardware-Interface; in der URDF stehen am Joint die *linearen* Limits.

Teil VIII

Dokumentationsmethodik

Kapitel 30

Vorgehensweise festhalten

Eine konsistente Dokumentation der Projekte schlägt folgende Struktur und Werkzeuge vor:

Versionskontrolle

Alles in **Git**. Ein Monorepo oder ein Repo je Projekt mit klarer **README**.

Verzeichnisstruktur

`/docs` (Theorie, Entscheidungen), `/design` (CAD, URDF, Frames), `/hardware` (Stückliste/BOM, Verdrahtung, Pinbelegung), `/software` (Pakete, Launch-Files, Parameter-YAMLs), `/calibration` (Kamera-.yaml), `/logs` (Tests, `ros2 bag`-Aufzeichnungen).

Entscheidungsprotokoll (ADR)

Kurze Notizen “Warum TMC2209 statt A4988?”, “Warum Jazzy statt Humble?” – spart in Monaten viel Grübeln.

Code-Doku

Doxygen (C++) bzw. Docstrings + **Sphinx** (Python); `rostdoc2` für ROS 2-Pakete.

Reproduzierbarkeit

Launch-Files und Parameter-YAMLs sind Teil der Doku – sie machen ein Experiment wiederholbar. `rqt_graph`-Screenshots je Projekt beilegen.

Nachweise

`ros2 bag` zeichnet reale Läufe auf; damit kannst du Vision/Kalman *offline* debuggen und Sim vs. Real vergleichen.

Empfohlene Reihenfolge der Umsetzung

1. Setup (Teil III) → 2. URDF aus CAD + in Gazebo bewegen (Teil IV) → 3. `ros2_control` in Sim, Trajektorie abfahren (Projekt 1 simuliert) → 4. Stereokamera real anbinden + kalibrieren (Teil VI) → 5. Kalman/Vision auf simulierten *und* realen Daten → 6. Hardware-Interface tauschen, ESP32 + TMC2209 real → 7. projektspezifische Logik (Tischtennis, Navigation, Greifen). Jeder Schritt ist für sich testbar – das ist der Sinn der `ros2_control`-Trennung.

Literaturverzeichnis

- [1] Rasmus Laurvig Haugaard und Anders Glent Buch. “SurfEmb: Dense and Continuous Correspondence Distributions for Object Pose Estimation with Learnt Surface Embeddings”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2022.
- [2] Yi Zhou u. a. “On the Continuity of Rotation Representations in Neural Networks”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019.
- [3] Radu Bogdan Rusu, Nico Blodow und Michael Beetz. “Fast Point Feature Histograms (FPFH) for 3D Registration”. In: *IEEE International Conference on Robotics and Automation (ICRA)*. 2009.
- [4] Paul J. Besl und Neil D. McKay. “A Method for Registration of 3-D Shapes”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 14.2 (1992), S. 239–256.
- [5] Jiaolong Yang u. a. “Go-ICP: A Globally Optimal Solution to 3D ICP Point-Set Registration”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 38.11 (2016), S. 2241–2254.
- [6] Charles R. Qi u. a. “PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017.
- [7] Charles R. Qi u. a. “PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017.
- [8] Tomáš Hodaň u. a. “BOP: Benchmark for 6D Object Pose Estimation”. In: *European Conference on Computer Vision (ECCV)*. 2018.
- [9] Emanuel Todorov, Tom Erez und Yuval Tassa. “MuJoCo: A physics engine for model-based control”. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2012, S. 5026–5033.
- [10] Google DeepMind. *MuJoCo Documentation*. 2024. URL: <https://mujoco.readthedocs.io/>.
- [11] NVIDIA. *NVIDIA Isaac Sim Documentation*. 2024. URL: <https://docs.isaacsim.omniverse.nvidia.com/>.
- [12] Mayank Mittal u. a. “Orbit: A Unified Simulation Framework for Interactive Robot Learning Environments”. In: *IEEE Robotics and Automation Letters (RA-L)*. 2023.